

Sistem de Analiză și Predicție a Traficului Aerian

Cuprins

1. [Introducere](#)
 - 1.1 Prezentarea setului de date
 - 1.2 Obiectivele proiectului
2. [Procesarea datelor cu Apache Spark](#)
 - 2.1 Inițializarea SparkSession
 - 2.2 Încărcarea și inspecția datelor
 - 2.3 Curățarea datelor (DataFrame API)
 - 2.4 Transformarea datelor (DataFrame API)
 - 2.5 Agregări cu DataFrame API
 - 2.6 Analiză cu Spark SQL
 - 2.7 Salvarea tabelului agregat
3. [Metode ML \(Machine Learning\)](#)
 - 3.1 Inițializare SparkSession și încărcare date
 - 3.2 ML 1 RandomForest Regression
 - 3.2.1 Enunțul problemei
 - 3.2.2 Justificarea alegerii metodei
 - 3.2.3 Explicarea soluției și a feature-urilor
 - 3.2.4 Pregătirea datelor pentru ML
 - 3.2.5 Construirea Pipeline-ului ML
 - 3.2.6 Split cronologic și Hyperparameter Tuning
 - 3.2.7 Aplicarea și evaluarea modelului
 - 3.2.8 Importanța feature-urilor și salvarea modelului
 - 3.3 ML 2 KMeans Clustering
 - 3.3.1 Enunțul problemei
 - 3.3.2 Justificarea alegerii metodei
 - 3.3.3 Explicarea soluției
 - 3.3.4 Calculul profilului per aeroport
 - 3.3.5 Alegerea numărului optim de clustere
 - 3.3.6 Antrenarea KMeans cu k=3
 - 3.3.7 Evaluarea KMeans și validarea clusterelor
4. [Data Pipeline](#)
5. [Funcție definită de utilizator \(UDF\) și Optimizarea hiperparametrilor](#)
6. [Metoda DL \(Deep Learning\)](#)
 - 6.1 Enunțul problemei

- 6.2 Justificarea alegerii metodei
 - 6.3 Explicarea soluției
 - 6.4 Pregătirea datelor pentru LSTM
 - 6.5 Construirea ferestrelor glisante
 - 6.6 Construirea și antrenarea modelului LSTM
 - 6.7 Evaluarea modelului LSTM
 - 6.8 Importanța feature-urilor pentru LSTM
7. Proces de streaming
- 7.1 Inițializarea SparkSession cu suport Kafka
 - 7.2 Definirea schemei și a zonelor geografice monitorizate
 - 7.3 Citirea din Kafka și procesarea streamului
 - 7.4 Funcția de procesare per batch (foreachBatch)
 - 7.5 Pornirea streaming-ului
 - 7.6 Oprirea streaming-ului
 - 7.7 Statistici despre topicul Kafka

Notebook 1 – Procesarea și analiza datelor

1. Introducere

1.1 Prezentarea setului de date

Setul de date utilizat în cadrul acestui proiect a fost obținut prin colectarea sistematică de informații despre traficul aerian, utilizând **OpenSky Network API** (<https://openskynetwork.github.io/opensky-api/python.html#examples>).

Având în vedere restricțiile privind numărul limitat de credite API oferite de OpenSky Network, procesul de colectare a fost implementat într-o manieră incrementală. Astfel, scriptul de colectare **concatenează datele noi** la fișierul `history_traffic.csv` existent, prevenind suprascrierea și asigurând acumularea unui volum istoric consistent, necesar pentru antrenarea modelelor de Machine Learning.

Setul de date procesat (format .csv) este disponibil în repository-ul GitHub al proiectului, în folderul /data, și poate fi accesat direct:

Link Github: <https://github.com/CaruntuRazvan/BigData-Sky-Traffic-Analytics/tree/main/data>.

Setul de date utilizat cuprinde informații colectate în intervalul **04-04-2026 – 05-07-2026**, acoperind **5 aeroporturi europene majore**:

- **LROP** - Henri Coandă, București (România)
- **EGLL** - Heathrow, Londra (Marea Britanie)
- **LFPG** - Charles de Gaulle, Paris (Franța)
- **EDDF** - Frankfurt (Germania)
- **EHAM** - Amsterdam Schiphol (Olanda)

Pentru fiecare aeroport au fost colectate atât **sosirile** cât și **plecărilor** pe parcursul mai multor zile consecutive, folosind endpoint-urile `/flights/arrival` și `/flights/departure` ale API-ului OpenSky.

Coloanele setului de date:

Coloană	Tip	Descriere
<code>icao24</code>	string	Adresa unică ICAO 24-bit a transponderului
<code>callsign</code>	string	Indicativul zborului (ex: DLH7WK)
<code>airport</code>	string	Codul ICAO al aeroportului monitorizat
<code>type</code>	string	Tipul zborului: <code>arrival</code> sau <code>departure</code>
<code>firstSeen</code>	long	Timestamp Unix al primei detecții
<code>lastSeen</code>	long	Timestamp Unix al ultimei detecții
<code>arrival_hour</code>	int	Ora sosirii/plecării (0-23)
<code>day_of_week</code>	int	Ziua săptămânii (0=Luni, 6=Duminică)
<code>estDepartureAirport</code>	string	Aeroportul de plecare estimat
<code>estArrivalAirport</code>	string	Aeroportul de sosire estimat
<code>date</code>	string	Data zborului (YYYY-MM-DD)

Exemplu de date (format CSV):

```
icao24,callsign,airport,type,firstSeen,lastSeen,arrival_hour,day_of_week,estDepartureAirport,estArrivalAirport,date
3c65cd,DLH7WK,LROP,arrival,1782586842,1782593625,20,5,EDDF,LROP,2026-06-27
471f62,WMT4GW,LROP,arrival,1782582120,1782593488,20,5,LEMD,LROP,2026-06-27
4a08e9,R0T384J,LROP,arrival,1782583944,1782593360,20,5,LFPG,LROP,2026-06-27
```

1.2 Obiectivele proiectului

Scopul lucrării este dezvoltarea unui sistem distribuit de procesare, analiză și predicție a traficului aerian. Obiectivele specifice sunt:

- **Ingestia și procesarea datelor Big Data:** Colectarea automată și curățarea unui volum mare de date (batch și stream) utilizând Apache Spark și Apache Kafka, cu gestionarea eficientă a resurselor API.
- **Analiza exploratorie (EDA):** Identificarea pattern-urilor operaționale prin Spark SQL, precum rutele frecvente, sezonabilitatea și intervalele orare de vârf din aeroporturi.
- **Predicție și modelare Machine Learning:**
 - **Regresie:** Estimarea numărului de zboruri pe oră utilizând algoritmul Random Forest.
 - **Clustering:** Segmentarea aeroporturilor în funcție de intensitatea traficului aerian folosind algoritmul K-Means.
 - **Deep Learning:** Prognozarea evoluției traficului aerian prin intermediul rețelelor neuronale de tip LSTM.
- **Monitorizare în timp real:** Implementarea unui flux de procesare live care detectează aeronavele aflate în proximitatea aeroporturilor (Geofencing), clasifică starea lor operațională (de exemplu: *Apropiere*, *Aterizare*) și generează rapoarte dinamice privind activitatea aeroportuară.

2. Procesarea datelor cu Apache Spark

2.1 Inițializarea SparkSession

Primul pas este crearea unei sesiuni Spark. Folosim `local[*]` pentru a utiliza toate core-urile disponibile pe mașina locală.

```
import os
import sys

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

print(f'Python path: {sys.executable}')
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, udf, when, trim, avg, dayofweek, count, to_date, round as
    spark_round,
    min as spark_min, max as spark_max
)
from pyspark.sql.types import StringType, IntegerType
import os

spark = SparkSession.builder \
    .appName('AirTrafficAnalysis') \
    .master('local[*]') \
    .getOrCreate()
```

```
spark.sparkContext.setLogLevel('WARN')
print(f'Spark version: {spark.version}')
print('SparkSession initializat cu succes!')
```

```
Python path: C:\Users\Lenovo\anaconda3\envs\spark_env\python.exe
Spark version: 3.5.3
SparkSession initializat cu succes!
```

2.2 Încărcarea și inspecția datelor

Încărcăm CSV-ul colectat din OpenSky Network și inspectăm structura acestuia: schema, numărul de înregistrări și primele rânduri.

```
CSV_PATH = os.path.join('..', 'data', 'history_traffic.csv')

df_raw = spark.read.csv(CSV_PATH, header=True, inferSchema=True)

print(f'Total inregistrari brute: {df_raw.count()}')
print(f'Numar coloane: {len(df_raw.columns)}')
print()
df_raw.printSchema()
```

```
Total inregistrari brute: 512977
Numar coloane: 11
```

```
root
|-- icao24: string (nullable = true)
|-- callsign: string (nullable = true)
|-- airport: string (nullable = true)
|-- type: string (nullable = true)
|-- firstSeen: integer (nullable = true)
|-- lastSeen: integer (nullable = true)
|-- arrival_hour: integer (nullable = true)
|-- day_of_week: integer (nullable = true)
|-- estDepartureAirport: string (nullable = true)
|-- estArrivalAirport: string (nullable = true)
|-- date: date (nullable = true)
```

```
# Primele 10 randuri pentru a inspecta datele
df_raw.show(10, truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+
|icao24|callsign|airport|type  |firstSeen|lastSeen|arrival_hour|
|day_of_week|estDepartureAirport|estArrivalAirport|date      |
+-----+-----+-----+-----+-----+-----+-----+
```

```

+-----+-----+-----+-----+-----+-----+-----+
|3c65cd|DLH7WK  |LROP  |arrival|1782586842|1782593625|20      |5
|EDDF   |           |LROP  |       |2026-06-27|          |
|471f62|WMT4GW  |LROP  |arrival|1782582120|1782593488|20      |5
|LEMD   |           |LROP  |       |2026-06-27|          |
|4a08e9|ROT384J |LROP  |arrival|1782583944|1782593360|20      |5
|LFPG   |           |LROP  |       |2026-06-27|          |
|4ca708|RYR3EM  |LROP  |arrival|1782586700|1782592638|20      |5
|EDDB   |           |LROP  |       |2026-06-27|          |
|48ada9|LOT639  |LROP  |arrival|1782587673|1782592501|20      |5
|EPWA   |           |LROP  |       |2026-06-27|          |
|39e687|AFR22KN |LROP  |arrival|1782583502|1782592373|20      |5
|LFPG   |           |LROP  |       |2026-06-27|          |
|4cafc2|RYR69TW |LROP  |arrival|1782582571|1782591988|20      |5
|EGSS   |           |LROP  |       |2026-06-27|          |
|4a0d12|AWG4151 |LROP  |arrival|1782586440|1782591547|20      |5
|NULL   |           |LROP  |       |2026-06-27|          |
|471f6e|WMT711  |LROP  |arrival|1782578310|1782590950|20      |5
|LEMG   |           |LROP  |       |2026-06-27|          |
|4ca857|RYR6EA  |LROP  |arrival|1782579140|1782590788|20      |5
|LEMD   |           |LROP  |       |2026-06-27|          |
+-----+-----+-----+-----+-----+-----+-----+
only showing top 10 rows

```

2.3 Curățarea datelor (DataFrame API)

Înainte de analiză, curățăm datele:

- Eliminăm spațiile din `callsign` (observate în datele brute: DLH7WK)
- Eliminăm rândurile cu `arrival_hour` sau `airport` null
- Eliminăm rândurile cu `estDepartureAirport` gol (zboruri fără origine cunoscută)
- Excluderea zborurilor în care aeroportul de plecare este identic cu cel de sosire (`estDepartureAirport` $\neq$ `estArrivalAirport`)
- Filtrăm `arrival_hour` să fie în intervalul valid [0, 23]
- Eliminăm duplicatele pe baza `icao24` + `date` + `airport` + `type`

```

df_clean = df_raw \
    .withColumn('callsign', trim(col('callsign'))) \
    .dropna(subset=['arrival_hour', 'airport', 'lastSeen',
'firstSeen']) \
    .filter(col('estDepartureAirport').isNotNull() &
(col('estDepartureAirport') != '')) \
    .filter(col('estArrivalAirport').isNotNull() &
(col('estArrivalAirport') != '')) \
    .filter(col('estDepartureAirport') != col('estArrivalAirport')) \

```

```

        .filter(col('arrival_hour').between(0, 23)) \
        .dropDuplicates(['icão24', 'date', 'airport', 'type'])

print(f'Inregistrari dupa curatare: {df_clean.count()}')
print(f'Inregistrari eliminate:      {df_raw.count() -
df_clean.count()}')

Inregistrari dupa curatare: 292899
Inregistrari eliminate:      220078

```

2.4 Transformarea datelor (DataFrame API)

Adăugăm coloane derivate utile pentru analiză și modelare:

- `flight_duration_min` - durata zborului în minute, calculată din `lastSeen` - `firstSeen`
- `is_weekend` - indicator boolean dacă zborul e în weekend (sâmbătă=5 sau duminică=6)
- `rush_period` - perioada zilei, calculată printr-o **funcție UDF definită de utilizator**

```

from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

@udf(returnType=StringType())
def classify_rush(hour):
    if hour is None: return 'Unknown'
    hour = int(hour)
    if 0 <= hour <= 5: return 'Off-Peak'
    elif 6 <= hour <= 9: return 'Morning Rush'
    elif 10 <= hour <= 15: return 'Midday'
    elif 16 <= hour <= 20: return 'Evening Rush'
    else: return 'Night'

# 1. Aplicam filtrul general (zboruri între 30m - 24)
df = df_clean \
    .withColumn('flight_duration_min', spark_round((col('lastSeen') -
col('firstSeen')) / 60, 1)) \
    .filter(
        (col('flight_duration_min').between(30, 1440)) &
        (col('estDepartureAirport') != col('estArrivalAirport'))
    ) \
    .withColumn('day_of_week', (dayofweek(to_date(col('date'), 'yyyy-
MM-dd')) + 5) % 7) \
    .withColumn('is_weekend', when(col('day_of_week') >= 5,
1).otherwise(0)) \
    .withColumn('rush_period', classify_rush(col('arrival_hour')))

print(f'Total randuri dupa transformari: {df.count()}')

```

```

print(f'Coloane noi adaugate: flight_duration_min, is_weekend,
rush_period, day_of_week')
df.select('callsign', 'airport', 'type', 'arrival_hour',
          'rush_period', 'is_weekend', 'flight_duration_min',
          'day_of_week').show(10)

```

Total randuri dupa transformari: 289129
Coloane noi adaugate: flight_duration_min, is_weekend, rush_period, day_of_week

callsign	airport	type	arrival_hour	rush_period	is_weekend	flight_duration_min	day_of_week
MSR778	EGLL	departure	19	Evening Rush	0	238.6	4
MSR786	EDDF	departure	18	Evening Rush	0	204.9	4
MSR780	EGLL	departure	1	Off-Peak	0	246.7	1
MSR802	LFPG	departure	0	Off-Peak	0	236.0	0
MSR784	EGLL	departure	13	Midday	1	249.7	6
MSR780	EGLL	departure	1	Off-Peak	0	271.3	2
MSR784	EGLL	departure	22	Night	0	822.1	0
MSR758	EHAM	departure	18	Evening Rush	0	246.2	2
MSR758	EHAM	departure	18	Evening Rush	0	243.9	0
MSR758	EHAM	departure	18	Evening Rush	0	240.7	2

only showing top 10 rows

2.5 Agregări cu DataFrame API

2.5.1 Numărul total de zboruri per aeroport și tip

Calculăm distribuția zborurilor per aeroport, separând sosirile de plecări, pentru a înțelege volumul de trafic al fiecărui aeroport.


```
df.groupBy('airport', 'type') \
  .agg(count('*').alias('total_zboruri')) \
  .orderBy('airport', 'type') \
  .show()
```

airport	type	total_zboruri
EDDF	arrival	29431
EDDF	departure	29221
EGLL	arrival	37217
EGLL	departure	37641
EHAM	arrival	33558
EHAM	departure	33517
LFPG	arrival	35417
LFPG	departure	36023
LROP	arrival	8555
LROP	departure	8549

2.5.2 Trafic mediu per oră per aeroport

Agregăm datele pentru a obține numărul mediu de zboruri per oră pentru fiecare aeroport. Acesta este **tabelul principal** care va fi folosit pentru antrenarea modelelor ML.

```
from pyspark.sql.functions import lag, quarter, countDistinct,
substring, sin, cos, to_date, col, stddev
from pyspark.sql import Window

w_lag = Window.partitionBy('airport', 'arrival_hour').orderBy('date')
w_7days = Window.partitionBy('airport', 'arrival_hour') \
  .orderBy('date') \
  .rowsBetween(-7, -1)

# Nr rute unice din ZIUA PRECEDENTA
w_yesterday = Window.partitionBy('airport', 'arrival_hour') \
  .orderBy('date') \
  .rowsBetween(-1, -1)

nr_route_daily = df.groupBy('airport', 'arrival_hour', 'date') \
  .agg(countDistinct('estDepartureAirport').alias('nr_
rute_unice_day'))

nr_companii_daily = df.withColumn('airline_code',
  substring(col('callsign'), 1, 3)) \
  .groupBy('airport', 'arrival_hour', 'date') \
  .agg(countDistinct('airline_code').alias('nr_comp
anii_day'))
```

```
# Adaugam la df_hourly si aplicam lag pentru a folosi doar date din trecut
df_hourly = (df
    .withColumn('day_of_week', (dayofweek(to_date(col('date')), 'yyyy-MM-dd')) + 5) % 7)
    .groupBy('airport', 'arrival_hour', 'day_of_week', 'is_weekend', 'date')
    .agg(count('*').alias('flights_count'))
    .join(nr_rute_daily, on=['airport', 'arrival_hour', 'date'], how='left')
    .join(nr_companii_daily, on=['airport', 'arrival_hour', 'date'], how='left')
    .withColumn('flights_yesterday', lag('flights_count', 1).over(w_lag))
    .withColumn('media_7zile', avg('flights_count').over(w_7days))
    .withColumn('quarter', quarter(to_date(col('date')), 'yyyy-MM-dd'))
    .withColumn('nr_rute_unice', lag('nr_rute_unice_day', 1).over(w_lag))
    .withColumn('nr_companii', lag('nr_companii_day', 1).over(w_lag))
    .withColumn('hour_sin', sin(col('arrival_hour') * 3.14159 / 12))
    .withColumn('hour_cos', cos(col('arrival_hour') * 3.14159 / 12))
    .withColumn('volatilitate_7zile', stddev('flights_count').over(w_7days))
    .fillna({'volatilitate_7zile': 0})
    .drop('nr_rute_unice_day', 'nr_companii_day')
    .orderBy('airport', 'date', 'arrival_hour')
)

print(f'df_hourly fara data leakage: {df_hourly.count()} randuri')
df_hourly.select('airport', 'arrival_hour', 'date', 'flights_count', 'flights_yesterday', 'nr_rute_unice', 'nr_companii').show(20)
```

df_hourly fara data leakage: 10851 randuri

airport	arrival_hour	date	flights_count	flights_yesterday	nr_rute_unice	nr_companii
EDDF	0	2026-04-04	4	NULL	NULL	NULL
EDDF	1	2026-04-04	3	NULL	NULL	NULL
EDDF	2	2026-04-04	4	NULL	NULL	NULL
EDDF	3	2026-04-04	17	NULL	NULL	NULL


```

        spark_round(spark_min('flight_duration_min'),
1).alias('durata_min_min'),
        spark_round(spark_max('flight_duration_min'),
1).alias('durata_max_min')
    ) \
    .orderBy('durata_medie_min', ascending=False) \
    .show()

```

```

+-----+-----+-----+-----+
|airport|durata_medie_min|durata_min_min|durata_max_min|
+-----+-----+-----+-----+
|   EGLL   |          301.4|          30.0|          1438.0|
|   EDDF   |          227.7|          30.0|          1438.9|
|   LFPG   |          224.3|          30.0|          1438.4|
|   EHAM   |          196.0|          30.0|          1439.2|
|   LROP   |          128.7|          30.0|          1439.7|
+-----+-----+-----+-----+

```

2.5.4 Cel mai scurt / lung zbor detectat pentru fiecare aeroport

```

# Zborul cel mai scurt per aeroport
print("=== Cel mai scurt zbor detectat per aeroport ===")
from pyspark.sql import Window
from pyspark.sql.functions import rank

w_min = Window.partitionBy('airport').orderBy('flight_duration_min')
df.withColumn('rank', rank().over(w_min)) \
    .filter(col('rank') == 1) \
    .select('airport', 'callsign', 'type', 'estDepartureAirport',
            'estArrivalAirport', 'flight_duration_min') \
    .orderBy('airport') \
    .show()

print("=== Cel mai lung zbor detectat per aeroport ===")
w_max =
Window.partitionBy('airport').orderBy(col('flight_duration_min').desc(
))
df.withColumn('rank', rank().over(w_max)) \
    .filter(col('rank') == 1) \
    .select('airport', 'callsign', 'type', 'estDepartureAirport',
            'estArrivalAirport', 'flight_duration_min') \
    .orderBy('airport') \
    .show()

=== Cel mai scurt zbor detectat per aeroport ===
+-----+-----+-----+-----+-----+
+-----+
|airport|callsign|      type|estDepartureAirport|estArrivalAirport|
flight_duration_min|

```

```

+-----+-----+-----+-----+-----+
+-----+
| EDDF| DLA8WU| arrival|          ELLX|          EDDF|
30.0|
| EDDF| LHX076| departure|        EDDF|        EDDL|
30.0|
| EDDF| DLH147| arrival|        EDDN|        EDDF|
30.0|
| EDDF| NJE413F| arrival|        EDDL|        EDDF|
30.0|
| EDDF| DLH1AN| arrival|        EDDS|        EDDF|
30.0|
| EDDF| DLH1AN| arrival|        EDDS|        EDDF|
30.0|
| EDDF| DLH1AN| arrival|        EDDS|        EDDF|
30.0|
| EDDF| DHXCG| departure|        EDDF|        EDGJ|
30.0|
| EDDF| DLH048| departure|        EDDF|        EDDV|
30.0|
| EDDF| DLH078| departure|        EDDF|        EDDL|
30.0|
| EDDF| DLH1AN| arrival|        EDDS|        EDDF|
30.0|
| EDDF| DLH5CW| arrival|        EDDP|        EDDF|
30.0|
| EDDF| ADN64E| departure|        EDDF|        EDDN|
30.0|
| EDDF| SWR1AM| departure|        EDDF|        LSZH|
30.0|
| EGLL| BAW9175| arrival|        EGFF|        EGLL|
30.0|
| EGLL| SXN19| arrival|        EGBW|        EGLL|
30.0|
| EGLL| SHT3H| arrival|        EGCC|        EGLL|
30.0|
| EGLL| SHT2B| departure|        EGLL|        EGCC|
30.0|
| EGLL| SHT2T| departure|        EGLL|        EGCC|
30.0|
| EGLL| SHT2B| departure|        EGLL|        EGCC|
30.0|
+-----+-----+-----+-----+-----+
+-----+
only showing top 20 rows

=== Cel mai lung zbor detectat per aeroport ===
+-----+-----+-----+-----+-----+
+-----+

```

airport	callsign	type	estDepartureAirport	estArrivalAirport	flight_duration_min
EDDF	CA01048	departure	EDDF	PANC	1438.9
EGLL	SVA117	arrival	VTBS	EGLL	1438.0
EHAM	CSG2543	departure	EHAM	PANC	1439.2
LFPG	ETH735	departure	LFPG	SBGR	1438.4
LROP	AAE141	arrival	VHHH	LROP	1439.7

2.5.5 Top rute (perechi departure → arrival) cele mai frecvente

Identificăm cele mai frecvente rute din setul de date, adică perechile de aeroporturi între care există cel mai mult trafic.

```
from pyspark.sql.functions import coalesce

df_routes = df.withColumn(
    'plecare',
    when(col('type') == 'arrival', col('estDepartureAirport'))
    .otherwise(col('airport'))
).withColumn(
    'sosire',
    when(col('type') == 'arrival', col('airport'))
    .otherwise(col('estArrivalAirport'))
).filter(
    col('plecare').isNotNull() & col('sosire').isNotNull()
).filter(
    col('plecare') != col('sosire')
)

df_routes.groupBy('plecare', 'sosire') \
    .agg(count('*').alias('frecventa')) \
    .orderBy('frecventa', ascending=False) \
    .show(15)
```

plecare	sosire	frecventa
KJFK	EGLL	1938
EGLL	KJFK	1918
EHAM	EGLL	1907

	LFPG	EDDF	1834	
	EGLL	EHAM	1754	
	EGLL	LFPG	1653	
	LFPG	EGLL	1597	
	EDDF	EGLL	1549	
	EDDF	LFPG	1549	
	EGLL	EDDF	1489	
	LFPG	EHAM	1457	
	EHAM	LFPG	1378	
	EHAM	EDDF	1179	
	LOWW	EDDF	1144	
	EDDF	EHAM	1139	

+-----+-----+
only showing top 15 rows

2.6 Analiză cu Spark SQL

Înregistrăm DataFrame-ul ca view temporar pentru a putea folosi Spark SQL.

2.6.1 Cele mai aglomerate ore per aeroport

Identificăm top 3 cele mai aglomerate ore pentru fiecare aeroport, calculând media zborurilor pe oră de-a lungul tuturor zilelor colectate.

```
df_hourly.createOrReplaceTempView('flights_hourly')
df.createOrReplaceTempView('flights')

from pyspark.sql.functions import rank
from pyspark.sql import Window

w =
Window.partitionBy('airport').orderBy(col('medie_zboruri_pe_ora').desc
())

spark.sql("""
    SELECT
        airport,
        arrival_hour,
        ROUND(AVG(flights_count), 1) AS medie_zboruri_pe_ora,
        MAX(flights_count)          AS maxim_zboruri
    FROM flights_hourly
    GROUP BY airport, arrival_hour
""").withColumn('rank', rank().over(w)) \
    .filter(col('rank') <= 3) \
    .orderBy('airport', 'rank') \
    .show()
```

airport	arrival_hour	medie_zboruri_pe_ora	maxim_zboruri	rank
EDDF	18	61.2	79	1
EDDF	17	56.5	73	2
EDDF	20	49.1	66	3
EGLL	20	74.9	93	1
EGLL	18	72.1	88	2
EGLL	17	66.8	78	3
EHAM	17	86.7	103	1
EHAM	20	72.1	89	2
EHAM	18	62.4	86	3
LFPG	20	66.9	86	1
LFPG	17	64.0	79	2
LFPG	18	61.2	77	3
LROP	16	15.1	26	1
LROP	20	13.3	21	2
LROP	14	13.1	18	3
LROP	18	13.1	21	3

2.6.2 Comparație trafic weekend vs weekday per aeroport

Analizăm dacă există diferențe semnificative între traficul din zilele lucrătoare și weekend pentru fiecare aeroport monitorizat.

```
spark.sql("""
    SELECT
        airport,
        CASE WHEN is_weekend = 1 THEN 'Weekend' ELSE 'Weekday' END AS
tip_zi,
        ROUND(AVG(flights_count), 1) AS medie_zboruri_pe_ora,
        SUM(flights_count) AS total_zboruri
    FROM flights_hourly
    GROUP BY airport, is_weekend
    ORDER BY airport, tip_zi
""").show()
```

airport	tip_zi	medie_zboruri_pe_ora	total_zboruri
EDDF	Weekday	26.7	41153
EDDF	Weekend	26.8	17499
EGLL	Weekday	34.0	53042
EGLL	Weekend	33.3	21816
EHAM	Weekday	30.6	47675
EHAM	Weekend	29.4	19400
LFPG	Weekday	33.1	51525

	LFPG Weekend	30.1	19915
	LROP Weekday	8.6	12096
	LROP Weekend	8.4	5008
+-----+-----+-----+-----+			

2.6.3 Top 10 rute internaționale cele mai frecvente

Identificăm cele mai frecvente rute internaționale din setul de date, excluzând zborurile unde aeroportul de plecare este același cu cel de sosire.

```
spark.sql("""
SELECT
    estDepartureAirport AS plecare,
    estArrivalAirport   AS sosire,
    COUNT(*)            AS nr_zboruri
FROM flights
WHERE estDepartureAirport IS NOT NULL
      AND estArrivalAirport IS NOT NULL
      AND estDepartureAirport != estArrivalAirport
GROUP BY estDepartureAirport, estArrivalAirport
ORDER BY nr_zboruri DESC
LIMIT 10
""").show()
```

+-----+-----+-----+-----+		
plecare	sosire	nr_zboruri
+-----+-----+-----+-----+		
KJFK	EGLL	1938
EGLL	KJFK	1918
EHAM	EGLL	1907
LFPG	EDDF	1834
EGLL	EHAM	1754
EGLL	LFPG	1653
LFPG	EGLL	1597
EDDF	EGLL	1549
EDDF	LFPG	1549
EGLL	EDDF	1489
+-----+-----+-----+-----+		

2.7 Salvarea tabelului agregat

Salvăm tabelul agregat `df_hourly` ca CSV pentru a fi folosit în notebooks-urile următoare (ML și DL).

```
output_path = os.path.join '..', 'data', 'hourly_traffic.csv')
df_hourly.coalesce(1) \
    .write \
    .csv(output_path, header=True, mode='overwrite')

print(f'Tabel agregat salvat in: {output_path}')
print(f'Total inregistrari: {df_hourly.count()}')

Tabel agregat salvat in: ..\data\hourly_traffic.csv
Total inregistrari: 10851

spark.stop()
print('SparkSession oprit!')

SparkSession oprit!
```

Notebook 2 - Metode de Machine Learning și Deep Learning

În acest notebook aplicăm metodele de Machine Learning și Deep Learning cerute:

Cerință	Metodă	Scop
ML #1	RandomForest Regression (Spark MLlib)	Predicția numărului de zboruri per oră per aeroport
ML #2	KMeans Clustering (Spark MLlib)	Gruparea aeroporturilor după profilul de trafic
DL	LSTM (TensorFlow/Keras)	Predicție time-series pe seria temporală a traficului

Date de intrare: tabelul agregat `hourly_traffic.csv` produs de Notebook 1, care conține pentru fiecare combinație aeroport + oră + dată: numărul de zboruri înregistrate și o serie de feature-uri derivate fără data leakage.

3. Metode ML (Machine Learning)

3.1. Inițializare SparkSession și încărcare date

Pornim o sesiune Spark în mod local și încărcăm tabelul agregat produs de Notebook 1. Setăm explicit variabilele de mediu pentru Python ca să evităm conflicte de versiuni între driver și worker pe Windows.

```

import os
import sys

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count

spark = SparkSession.builder \
    .appName('AirTrafficML') \
    .master('local[*]') \
    .getOrCreate()

spark.sparkContext.setLogLevel('WARN')
print(f'Spark version: {spark.version}')

Spark version: 3.5.3

```

Încărcăm tabelul agregat `hourly_traffic.csv` generat în Notebook 1. Acesta conține câte zboruri au fost înregistrate per aeroport, per oră și per dată, împreună cu feature-urile derivate: traficul din ziua precedentă, media ultimelor 7 zile, numărul de rute și companii unice, volatilitatea traficului și reprezentările circulare ale orei.

```

# tabelul agregat produs de Notebook 1
df_hourly = spark.read.csv(
    os.path.join('.', 'data', 'hourly_traffic.csv'),
    header=True,
    inferSchema=True
)

print(f'Inregistrari incarcate: {df_hourly.count()}')
df_hourly.printSchema()
df_hourly.show(2)

Inregistrari incarcate: 10851
root
|-- airport: string (nullable = true)
|-- arrival_hour: integer (nullable = true)
|-- date: date (nullable = true)
|-- day_of_week: integer (nullable = true)
|-- is_weekend: integer (nullable = true)
|-- flights_count: integer (nullable = true)
|-- flights_yesterday: integer (nullable = true)
|-- media_7zile: double (nullable = true)
|-- quarter: integer (nullable = true)
|-- nr_rute_unice: integer (nullable = true)
|-- nr_companii: integer (nullable = true)
|-- hour_sin: double (nullable = true)
|-- hour_cos: double (nullable = true)

```

```
|-- volatilitate_7zile: double (nullable = true)

+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
|airport|arrival_hour|      date|day_of_week|is_weekend|flights_count|
flights_yesterday|media_7zile|quarter|nr_rute_unice|nr_companii|
hour_sin|      hour_cos|volatilitate_7zile|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
|  EDDF|      0|2026-04-04|      5|      1|      4|
NULL|      NULL|      2|      NULL|      NULL|      0.0|
1.0|      0.0|
|  EDDF|      1|2026-04-04|      5|      1|      3|
NULL|      NULL|      2|      NULL|      NULL|0.2588188315049383|
0.9659258835223427|      0.0|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
only showing top 2 rows
```

3.2 ML 1 RandomForest Regression

3.2.1 Enunțul problemei

Dorim să prezicăm **numărul de zboruri** (`flights_count`) care vor fi înregistrate la un aeroport într-o anumită oră a zilei. Modelul trebuie să învețe pattern-urile istorice de trafic și să generalizeze pentru ore viitoare nevăzute în antrenare.

3.2.2 Justificarea alegerii metodei

RandomForest Regression este potrivit pentru această problemă din mai multe motive:

- Traficul aerian are relații **non-liniare** cu ora din zi — vârfuri dimineața (6-9) și seara (16-20), nu o creștere uniformă
- Este robust la date cu zgomot și nu necesită ca toate feature-urile să fie pe aceeași scală
- Oferă **importanța feature-urilor**, utilă pentru interpretarea și validarea modelului
- Rezistă la overfitting prin agregarea predicțiilor mai multor arbori de decizie independenți

3.2.3 Explicarea soluției și a feature-urilor

Modelul primește ca input următoarele feature-uri, toate calculate **fără data leakage** (folosind doar informații disponibile la momentul predicției):

Feature	Descriere	Justificare
<code>airport_idx</code>	Codul aeroportului (indexat numeric)	Fiecare aeroport are un volum de bază diferit
<code>arrival_hour</code>	Ora din zi (0-23)	Pattern-ul orar principal
<code>hour_sin, hour_cos</code>	Reprezentare circulară a orei	Ora 23 și ora 0 sunt adiacente — sin/cos captează asta
<code>day_of_week</code>	Ziua săptămânii (0=Luni)	Pattern-urile diferă între zilele lucrătoare
<code>is_weekend</code>	1 dacă sâmbătă/duminică	Weekend are trafic diferit față de weekday
<code>flights_yesterday</code>	Traficul de ieri la aceeași oră	<code>lag(1)</code> — folosește doar date din trecut
<code>media_7zile</code>	Media ultimelor 7 zile la aceeași oră	<code>rowsBetween(-7, -1)</code> — fără data leakage
<code>volatilitate_7zile</code>	Abaterea standard a ultimelor 7 zile	Indică cât de predictibil este traficul la acea oră
<code>nr_rute_unice</code>	Numărul de rute unice de ieri	Calculat din ziua precedentă — fără data leakage
<code>nr_companii</code>	Numărul de companii aeriene de ieri	Calculat din ziua precedentă — fără data leakage
<code>quarter</code>	Trimestrul anului	Captează sezonalitate pe termen lung

3.2.4 Pregătirea datelor pentru ML

Eliminăm înregistrările cu valori lipsă în coloanele folosite ca feature-uri. Primele zile din dataset nu au `flights_yesterday` sau `media_7zile` (nu există date anterioare pentru ele), deci sunt excluse automat.

```
df_ml = df_hourly \
    .dropna(subset=['airport', 'arrival_hour', 'day_of_week',
                    'is_weekend', 'flights_count',
                    'flights_yesterday',
                    'media_7zile', 'volatilitate_7zile',
                    'nr_rute_unice', 'nr_companii', 'quarter'])

print(f'Inregistrari pentru ML: {df_ml.count()}')
df_ml.select('airport', 'arrival_hour', 'flights_count',
             'flights_yesterday', 'media_7zile', 'volatilitate_7zile',
             'nr_rute_unice', 'nr_companii', 'quarter').show(10)
```

Inregistrari pentru ML: 10731

```
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|airport|arrival_hour|flights_count|flights_yesterday|media_7zile|
```

volatilitate_7zile	nr_rute_unice	nr_companii	quarter	
EDDF	0	5	4	4.0
0.0	1	2	2	
EDDF	1	2	3	3.0
0.0	1	3	2	
EDDF	2	5	4	4.0
0.0	1	4	2	
EDDF	3	11	17	17.0
0.0	12	11	2	
EDDF	4	12	12	12.0
0.0	12	8	2	
EDDF	5	18	18	18.0
0.0	11	10	2	
EDDF	6	30	25	25.0
0.0	17	12	2	
EDDF	7	29	31	31.0
0.0	17	21	2	
EDDF	8	25	20	20.0
0.0	16	14	2	
EDDF	9	21	22	22.0
0.0	11	16	2	

only showing top 10 rows

3.2.5 Construirea Pipeline-ului ML

Folosim un **Pipeline Spark MLlib** care automatizează toți pașii de preprocesare și modelare într-un singur obiect reutilizabil. Pipeline-ul conține 4 etape în ordine:

1. **StringIndexer** — transformă numele aeroportului (șir de caractere) în index numeric, necesar pentru algoritmi ML care lucrează cu valori numerice
2. **VectorAssembler** — combină toate feature-urile individuale într-un singur vector dens, formatul cerut de Spark MLlib
3. **StandardScaler** — normalizează fiecare feature la medie 0 și deviație standard 1, pentru stabilitate numerică
4. **RandomForestRegressor** — modelul de regresie propriu-zis

```
# Cerinta 4: Data Pipeline
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, VectorAssembler,
StandardScaler
from pyspark.ml.regression import RandomForestRegressor

# Pas 1: StringIndexer – transforma numele aeroportului in index
numeric
```

```

indexer = StringIndexer(
    inputCol='airport',
    outputCol='airport_idx',
    handleInvalid='keep'
)

# Pas 2: VectorAssembler – combina toate feature-urile intr-un vector
assembler = VectorAssembler(
    inputCols=[
        'airport_idx',
        'arrival_hour',
        'hour_sin',
        'hour_cos',
        'day_of_week',
        'is_weekend',
        'quarter',
        'flights_yesterday',
        'media_7zile',
        'volatilitate_7zile',
        'nr_rute_unice',
        'nr_companii'
    ],
    outputCol='features_raw'
)

# Pas 3: StandardScaler – normalizeaza valorile
scaler = StandardScaler(
    inputCol='features_raw',
    outputCol='features',
    withMean=True,
    withStd=True
)

# Pas 4: RandomForestRegressor – modelul
rf = RandomForestRegressor(
    labelCol='flights_count',
    featuresCol='features',
    seed=42
)

# Pipeline
pipeline = Pipeline(stages=[indexer, assembler, scaler, rf])

print('Pipeline creat: StringIndexer → VectorAssembler →
StandardScaler → RandomForest')

Pipeline creat: StringIndexer → VectorAssembler → StandardScaler →
RandomForest

```

3.2.6 Split cronologic și Hyperparameter Tuning

Pentru date de tip time-series, **split-ul aleator este incorect** deoarece modelul ar putea vedea date din viitor în antrenare. Folosim un **split cronologic**: antrenăm pe primele 80% din zile (cele mai vechi) și testăm pe ultimele 20% (cele mai recente). Astfel simulăm un scenariu realist în care modelul prezice zile pe care nu le-a văzut niciodată.

Optimizarea hiperparametrilor (Cerința 5) se face prin `CrossValidator` cu `ParamGridBuilder`. Testăm combinații de:

- `numTrees` — numărul de arbori din pădure
- `maxDepth` — adâncimea maximă a fiecărui arbore
- `minInstancesPerNode` — numărul minim de exemple per nod frunză (regularizare)

`CrossValidator` antrenează câte un model pentru fiecare combinație pe 3 fold-uri de validare încrucișată și alege automat combinația cu cel mai mic RMSE.

```
# Cerinta 5: Hyperparameter Tuning
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
from pyspark.ml.evaluation import RegressionEvaluator

# Split cronologic - antrenam pe trecut, testam pe viitor
dates = [r.date for r in
df_ml.select('date').distinct().orderBy('date').collect()]
split_date = dates[int(len(dates) * 0.8)]

train_df = df_ml.filter(col('date') < split_date)
test_df = df_ml.filter(col('date') >= split_date)

print(f'Split cronologic la data: {split_date}')
print(f'Train: {train_df.count()} randuri | Test: {test_df.count()}
randuri')
print(f'Train acoperă: {dates[0]} → {split_date}')
print(f'Test acoperă: {split_date} → {dates[-1]}')

paramGrid = ParamGridBuilder() \
    .addGrid(rf.numTrees, [100]) \
    .addGrid(rf.maxDepth, [10, 12]) \
    .addGrid(rf.minInstancesPerNode, [5, 10]) \
    .build()

evaluator = RegressionEvaluator(labelCol='flights_count',
predictionCol='prediction', metricName='rmse')

cv = CrossValidator(estimator=pipeline, estimatorParamMaps=paramGrid,
                    evaluator=evaluator, numFolds=3, seed=42)

print('Incep antrenarea (4 combinatii x 3 folduri = 12 modele)...')
cv_model = cv.fit(train_df)
print('Antrenare finalizata!')
```



```
Split cronologic la data: 2026-06-17
Train: 8558 randuri | Test: 2173 randuri
Train acoperă: 2026-04-05 → 2026-06-17
Test acoperă: 2026-06-17 → 2026-07-05
Incep antrenarea (4 combinatii x 3 folduri = 12 modele)...
Antrenare finalizata!
```

3.2.7 Aplicarea și evaluarea modelului

Aplicăm modelul pe setul de test (date din ultimele 20% zile, nevăzute în antrenare) și calculăm trei metrice de evaluare:

- **RMSE** (Root Mean Squared Error) — penalizează mai mult erorile mari
- **MAE** (Mean Absolute Error)
- **R²** — coeficientul de determinare, indică ce procent din variația datelor este explicat de model (1.0 = perfect)

Afișăm și predicțiile concrete pentru aeroportul București (LROP) pentru a verifica calitativ comportamentul modelului.

```
# Evaluarea modelului
from pyspark.sql import functions as F
from pyspark.sql.functions import round as spark_round
predictions = cv_model.transform(test_df)

rmse = evaluator.evaluate(predictions)
mae = RegressionEvaluator(
    labelCol='flights_count', predictionCol='prediction',
    metricName='mae'
).evaluate(predictions)
r2 = RegressionEvaluator(
    labelCol='flights_count', predictionCol='prediction',
    metricName='r2'
).evaluate(predictions)

print(f'\n=== Rezultate===')
print(f' RMSE: {rmse:.2f} zboruri')
print(f' MAE: {mae:.2f} zboruri')
print(f' R2: {r2:.4f}')

# Cei mai buni hiperparametri
best_rf = cv_model.bestModel.stages[-1]
print(f'\n Cei mai buni hiperparametri:')
print(f' numTrees: {best_rf.getNumTrees}')
print(f' maxDepth: {best_rf.getOrDefault("maxDepth")}')

print('\n Exemple predictii vs valori reale:')
predictions.select('airport', 'arrival_hour', 'day_of_week',
                    'flights_count', 'prediction') \
    .show(30)
```

```
# Predictii pentru LROP (Bucuresti)
print('\n=== Predictii LROP (Bucuresti) - profil orar complet ===')
predictions.filter(col('airport') == 'LROP') \
    .select('airport', 'arrival_hour', 'day_of_week',
            'flights_count', 'prediction') \
    .withColumn('prediction', spark_round(col('prediction'),
1)) \
    .orderBy('day_of_week', 'arrival_hour') \
    .show(30)

print('\n=== Corelatia predictie-realitate per aeroport ===')
predictions.groupBy('airport').agg(
    F.expr('corr(flights_count, prediction)').alias('corr_approx')
).show()
```

=== Rezultate===

RMSE: 8.13 zboruri
MAE: 4.40 zboruri
R2: 0.8404

Cei mai buni hiperparametri:

numTrees: 100
maxDepth: 12

Exemple predictii vs valori reale:

airport	arrival_hour	day_of_week	flights_count	prediction
EDDF	0	2	4	5.454813431274328
EDDF	1	2	3	2.448647598609407
EDDF	2	2	1	2.0292811483408726
EDDF	3	2	16	14.339686175061656
EDDF	4	2	20	18.163882483991305
EDDF	5	2	21	21.097416511535794
EDDF	6	2	26	26.751504209020478
EDDF	7	2	40	36.7678942464307
EDDF	8	2	27	27.27446908288895
EDDF	9	2	18	25.991426257727923
EDDF	10	2	27	25.023157488609172
EDDF	11	2	23	25.0597538224309
EDDF	12	2	24	21.349586568005396
EDDF	13	2	31	25.601436116243118
EDDF	14	2	27	32.679311038297094
EDDF	15	2	40	35.386470426814576
EDDF	16	2	48	50.153157681026094
EDDF	17	2	64	56.80458709401991
EDDF	18	2	61	65.2271183819375
EDDF	19	2	44	52.4371904012591

EDDF	20	2	50	52.02518587541808
EDDF	21	2	42	41.34292984893169
EDDF	22	2	9	10.601323477374045
EDDF	23	2	10	10.204383904041181
EDDF	0	3	5	5.382215018462083
EDDF	1	3	3	2.902305115863361
EDDF	2	3	1	1.9821991113735118
EDDF	3	3	17	13.939120039740205
EDDF	4	3	19	17.456132869029364
EDDF	5	3	20	20.28162513812043

only showing top 30 rows

=== Predictii LROP (Bucuresti) - profil orar complet ===

airport	arrival_hour	day_of_week	flights_count	prediction
LROP	0	0	3	2.1
LROP	0	0	2	1.6
LROP	1	0	3	1.5
LROP	2	0	1	1.5
LROP	2	0	3	1.4
LROP	3	0	1	1.5
LROP	4	0	4	3.3
LROP	4	0	7	4.2
LROP	5	0	11	8.7
LROP	5	0	9	8.9
LROP	6	0	7	7.3
LROP	6	0	8	6.1
LROP	7	0	6	6.1
LROP	7	0	5	5.0
LROP	8	0	8	9.2
LROP	8	0	4	8.2
LROP	9	0	6	8.1
LROP	9	0	5	7.4
LROP	10	0	14	9.1
LROP	10	0	13	9.4
LROP	11	0	11	10.0
LROP	11	0	9	9.9
LROP	12	0	11	13.0
LROP	12	0	10	10.0
LROP	13	0	6	9.8
LROP	13	0	10	9.7
LROP	14	0	16	12.8
LROP	14	0	13	13.9
LROP	15	0	9	13.3
LROP	15	0	12	12.8

only showing top 30 rows

=== Corelatia predictie-realitate per aeroport ===

airport	corr_approx
LFPG	0.8709281489660875
EGLL	0.9206542242213971
EDDF	0.8788130895716079
EHAM	0.9121446435203306
LROP	0.832150795941823

3.2.8 Importanța feature-urilor și salvarea modelului

RandomForest oferă o măsură a importanței fiecărui feature — cât de mult contribuie la reducerea erorii de predicție. Această analiză validează că feature-urile alese sunt relevante și că modelul a învățat pattern-uri logice:

- `media_7zile` și `flights_yesterday` cu importanță mare confirmă că **traficul recent este cel mai bun predictor** al traficului viitor
- `nr_companii` și `nr_rute_unice` cu importanță semnificativă confirmă că **diversitatea operatorilor** indică volumul de trafic
- `hour_sin` mai importante decât `arrival_hour` brut confirmă că **reprezentarea circulară a orei** este superioară

```
import pandas as pd

feature_names = [
    'airport_idx',
    'arrival_hour',
    'hour_sin',
    'hour_cos',
    'day_of_week',
    'is_weekend',
    'quarter',
    'flights_yesterday',
    'media_7zile',
    'volatilitate_7zile',
    'nr_rute_unice',
    'nr_companii'
]

feature_importances = best_rf.featureImportances.toArray()

fi_df = pd.DataFrame({
    'feature': feature_names,
    'importance': feature_importances
}).sort_values('importance', ascending=False)
```

```

print('\n=== Importanta Feature-urilor ===')
print(fi_df.to_string(index=False))

model_path = os.path.abspath(os.path.join '..', 'models', 'rf_model'))
os.makedirs(os.path.dirname(model_path), exist_ok=True)
model_path_spark = model_path.replace('\\', '/')
cv_model.bestModel.write().overwrite().save(model_path_spark)
print(f'\nModel salvat in: {model_path_spark}')

```

```

=== Importanta Feature-urilor ===

```

feature	importance
media_7zile	0.398688
flights_yesterday	0.237003
nr_companii	0.134487
nr_rute_unice	0.080477
hour_sin	0.052034
arrival_hour	0.034870
volatilitate_7zile	0.028740
hour_cos	0.015550
airport_idx	0.010931
day_of_week	0.005270
is_weekend	0.001950
quarter	0.000000

```

Model salvat in: D:/Facultate/master/an1/sem2/Big
Data/project/models/rf_model

```

3.3 ML 2 KMeans Clustering

3.3.1 Enunțul problemei

Dorim să **grupăm cele 5 aeroporturi** monitorizate după profilul lor de trafic, fără a folosi etichete predefinite. Scopul este să descoperim dacă există tipare naturale - de exemplu hub-uri mari cu trafic intens și variabil față de aeroporturi regionale cu volum redus și mai stabil.

3.3.2 Justificarea alegerii metodei

KMeans Clustering este potrivit deoarece:

- Nu avem etichete predefinite pentru tipul aeroportului - KMeans le descoperă din date
- Fiecare aeroport poate fi caracterizat prin câteva metrici agregate (trafic maxim, medie, variabilitate)
- KMeans este eficient și rezultatele sunt ușor interpretabile prin analiza centroizilor
- Numărul mic de aeroporturi (5) face metoda foarte tractabilă

3.3.3 Explicarea soluției

Caracterizăm fiecare aeroport prin două dimensiuni:

- **max_traffic** - traficul maxim înregistrat (indică capacitatea de vârf)
- **coef_variatie** - coeficientul de variație = deviație standard / medie (indică cât de impredictibil este traficul)

Aplicăm KMeans cu k=3 clustere pe aceste două dimensiuni după standardizare.

3.3.4 Calculul profilului per aeroport

Agregăm toate înregistrările per aeroport pentru a obține metricile de profil. Coeficientul de variație (deviație standard / medie) este o măsură normalizată a variabilității — un aeroport cu coef_variatie mare are trafic impredictibil, iar unul cu valoare mică are trafic regulat și ușor de anticipat.

```
from pyspark.sql.functions import max, min, stddev, expr, when

df_profile = df_hourly.groupBy('airport') \
    .agg(
        max('flights_count').alias('max_traffic'),
        avg('flights_count').alias('avg_traffic'),
        (stddev('flights_count') /
         avg('flights_count')).alias('coef_variatie')
    ).fillna(0)

print('Profil per aeroport:')
df_profile.orderBy('avg_traffic', ascending=False).show()
```

Profil per aeroport:

airport	max_traffic	avg_traffic	coef_variatie
EGLL	93	33.795936794582396	0.5898526829815255
LFPG	86	32.18018018018018	0.5772876136867164
EHAM	103	30.254848894903024	0.747233450845668
EDDF	79	26.75729927007299	0.6506415478609282
LROP	26	8.52217239661186	0.5513421442229648

3.3.5 Alegerea numărului optim de clustere

Înainte de a aplica KMeans cu un k fix, folosim **Metoda Cotului (Elbow Method)** pentru a determina numărul optim de clustere. Antrenăm KMeans pentru k de la 2 la 8 și vizualizăm costul (Within Set Sum of Squared Errors). Punctul unde costul începe să scadă mai lent (cotul graficului) indică k-ul optim.

```

import matplotlib.pyplot as plt
import numpy as np
from pyspark.ml.clustering import KMeans
from pyspark.ml.feature import VectorAssembler, StandardScaler

# vectorul de features pentru KMeans
assembler_km = VectorAssembler(
    inputCols=['max_traffic', 'coef_variatie'],
    outputCol='features_vec'
)
df_vec = assembler_km.transform(df_profile)

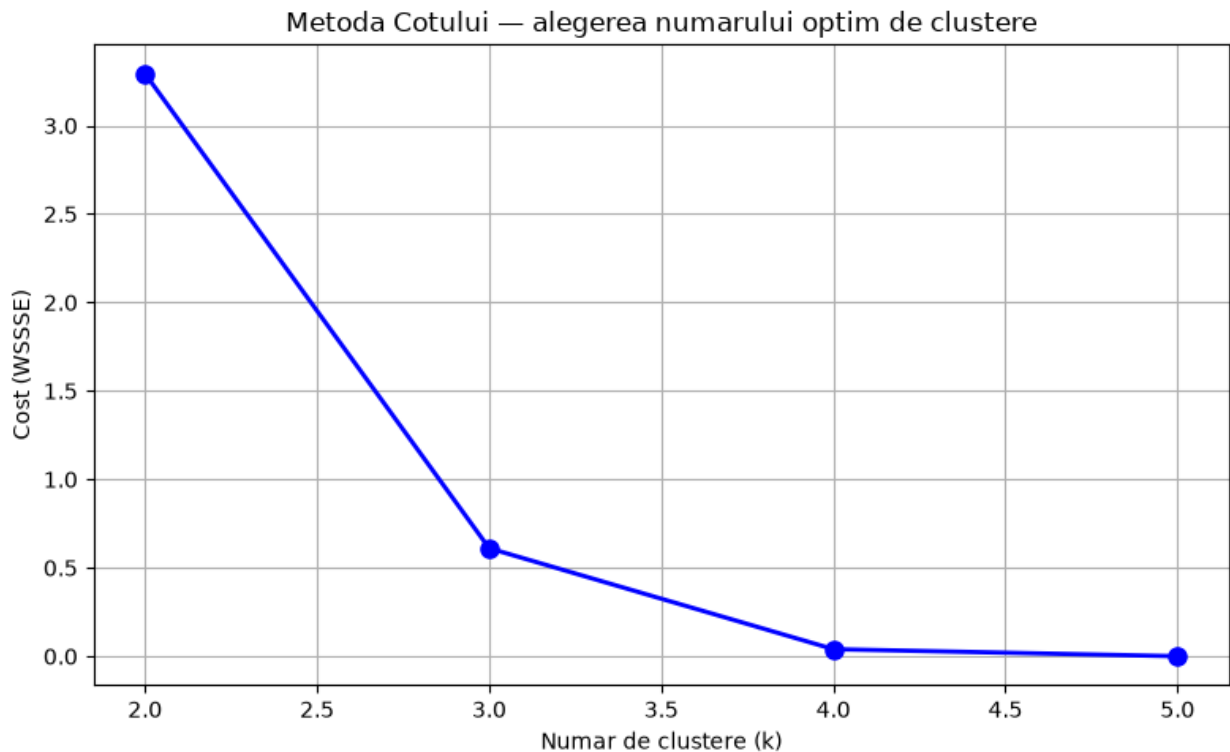
scaler_km = StandardScaler(inputCol='features_vec',
    outputCol='features', withStd=True, withMean=True)
df_scaled = scaler_km.fit(df_vec).transform(df_vec)

# Metoda Cotului
costs = []
k_range = range(2, 6)

for k in k_range:
    km = KMeans(k=k, featuresCol='features', seed=42)
    model_k = km.fit(df_scaled)
    costs.append(model_k.summary.trainingCost)

plt.figure(figsize=(8, 5))
plt.plot(k_range, costs, 'bo-', linewidth=2, markersize=8)
plt.xlabel('Numar de clustere (k)')
plt.ylabel('Cost (WSSSE)')
plt.title('Metoda Cotului – alegerea numarului optim de clustere')
plt.grid(True)
plt.tight_layout()
plt.show()

```



3.3.6 Antrenarea KMeans cu k=3

Pe baza metodei cotului, alegem k=3 clustere. Antrenăm modelul KMeans pe profilul standardizat al aeroporturilor și afișăm apartenența fiecărui aeroport la un cluster, împreună cu interpretarea fiecărui grup.

```
# KMeans cu k=3
kmeans = KMeans(k=3, featuresCol='features', predictionCol='cluster',
seed=42)
km_model = kmeans.fit(df_scaled)

df_clustered = km_model.transform(df_scaled).select('airport',
'cluster')

print('=== Rezultate KMeans Clustering ===')
df_clustered.show()

# Interpretarea clusterelor
print('=== Interpretarea clusterelor ===')
df_clustered.select('airport', 'cluster') \
    .withColumn('tip_aeroport',
        when(col('cluster') == 0, 'Hub Major - trafic intens
continuu')
        .when(col('cluster') == 1, 'Aeroport Regional - varfuri
clare'))
```



```

        .otherwise('Hub Major - trafic fluctuant')) \
        .show(truncate=False)

=== Rezultate KMeans Clustering ===
+-----+-----+
|airport|cluster|
+-----+-----+
|  LFPG |      0 |
|  EGLL |      0 |
|  EDDF |      0 |
|  EHAM |      2 |
|  LROP |      1 |
+-----+-----+

=== Interpretarea clusterelor ===
+-----+-----+-----+
|airport|cluster|tip_aeroport|
+-----+-----+-----+
|LFPG   |0      |Hub Major - trafic intens continuu|
|EGLL   |0      |Hub Major - trafic intens continuu|
|EDDF   |0      |Hub Major - trafic intens continuu|
|EHAM   |2      |Hub Major - trafic fluctuant      |
|LROP   |1      |Aeroport Regional - varfuri clare |
+-----+-----+-----+

```

3.3.7 Evaluarea KMeans și validarea clusterelor

Evaluăm calitatea clusterelor prin:

1. **Silhouette Score** - măsoară cât de bine separate sunt clusterele (valori aproape de 1.0 indică cluster clare)
2. **Analiza centrozilor** - valorile medii per cluster ne permit să interpretăm ce caracterizează fiecare grup
3. **Eroarea de predicție per cluster** - verificăm dacă gruparea aeroporturilor corelează cu precizia modelului RF

```

from pyspark.ml.evaluation import ClusteringEvaluator

# Silhouette Score
evaluator_km = ClusteringEvaluator(
    featuresCol='features',
    predictionCol='cluster',
    metricName='silhouette'
)
silhouette_score = evaluator_km.evaluate(df_clustered.join(df_scaled,
'airport'))
print(f'Silhouette Score: {silhouette_score:.4f}')

# Centrozii clusterelor

```

```

print('\n=== Centroizii clusterelor ===')
centers = km_model.clusterCenters()
for i, center in enumerate(centers):
    print(f'Cluster {i}: {center}')

Silhouette Score: 0.4725

=== Centroizii clusterelor ===
Cluster 0: [ 0.2859843 -0.22152189]
Cluster 1: [-1.70925498 -0.91868857]
Cluster 2: [0.85130209 1.58325424]

from pyspark.sql.functions import abs

if 'df_final_insight' not in globals():
    print("Se execută join-ul...")
    df_final_insight = df_clustered.join(df_profile, on='airport',
    how='left')
else:
    print("df_final_insight există deja -> skip.")

print('=== Profil detaliat per cluster ===')
df_final_insight.orderBy('cluster').show()

# Eroarea medie de predicție per cluster
if 'cluster' in df_ml.columns:
    df_ml = df_ml.drop('cluster')
df_ml_cu_cluster = df_ml.join(df_clustered, on='airport', how='left')

predictions_new = cv_model.transform(df_ml_cu_cluster)

predictions_new = predictions_new.withColumn(
    'eroare_absoluta', abs(col('flights_count') - col('prediction'))
)

print('\nEroarea medie de predicție per tip de aeroport (cluster):')
predictions_new.groupBy('cluster').avg('eroare_absoluta').orderBy('cluster').show()

```

Se execută join-ul...

=== Profil detaliat per cluster ===

airport	cluster	max_traffic	avg_traffic	coef_variatie
LFPG	0	86	32.18018018018018	0.5772876136867164
EGLL	0	93	33.795936794582396	0.5898526829815255
EDDF	0	79	26.75729927007299	0.6506415478609282
LROP	1	26	8.52217239661186	0.5513421442229648
EHAM	2	103	30.254848894903024	0.747233450845668

Eroarea medie de predicție per tip de aeroport (cluster):

```
+-----+-----+
|cluster|avg(eroare_absoluta)|
+-----+-----+
|      0|      3.383351928524491|
|      1|      1.659148236253205|
|      2|      3.1753962103495605|
+-----+-----+
```

4. Data Pipeline

Pentru detalii despre implementarea fluxului, vezi [secțiunea 3.2.5](#).

5. Funcție definită de utilizator(UDF) și Optimizarea hiperparametrilor

Pentru detalii despre cum am definit și utilizat funcția personalizată (UDF) în procesul de transformare, vezi [secțiunea 2.4](#). În ceea ce privește procesul de optimizare a hiperparametrilor pentru modelul de regresie, acesta este detaliat în [secțiunea 3.2.6](#).

6. Metoda DL (Deep Learning)

6.1 Enunțul problemei

Dorim să prezicăm **traficul din ora curentă** pentru aeroportul Heathrow (EGLL) pe baza ultimelor 24 de ore de trafic. Spre deosebire de RandomForest care tratează fiecare observație independent, modelul LSTM exploatează **dependențele temporale** - modul în care traficul din orele anterioare influențează traficul din ora prezentă.

6.2 Justificarea alegerii metodei

LSTM (Long Short-Term Memory) este o arhitectură de rețele neuronale recurente special proiectată pentru date secvențiale:

- Captează **dependențe pe termen lung** în serii temporale - de exemplu, un vârf de trafic dimineața poate fi urmat predictibil de o scădere la prânz
- Mecanismul de **gating** (forget gate, input gate, output gate) permite rețelei să decidă ce informații din trecut să rețină și ce să uite
- Este superior față de RandomForest pentru date cu **ordine temporală importantă**, unde context-ul secvențial contează

6.3 Explicarea soluției

Construim ferestre glisante de 24 de ore: pentru fiecare moment t , folosim feature-urile din orele $t-24$ până la $t-1$ ca input și prezicăm `flights_count` la momentul t . Rețeaua LSTM procesează secvența și produce o predicție scalară.

6.4 Pregătirea datelor pentru LSTM

Extragem seria temporală pentru EGLL și sortăm cronologic. Împărțim seria brută 80/20 (cronologic) înainte de scalare, pentru a evita data leakage: `MinMaxScaler` este calibrat (fit) doar pe segmentul de antrenare, apoi aplicat (transform) și pe segmentul de test. Pentru ca primele ferestre din test să aibă context istoric, păstrăm ultimele 24 de ore din train ca "punte" la începutul segmentului de test.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# extragem pt aeroportul EGLL
df_egll_full = df_hourly.filter(col('airport') == 'EGLL') \
    .dropna(subset=['flights_yesterday', 'media_7zile',
                    'nr_companii',
                    'volatilitate_7zile']) \
    .orderBy('date', 'arrival_hour') \
    .toPandas()

print(f'Inregistrari EGLL: {len(df_egll_full)}')

features = ['flights_count', 'flights_yesterday', 'media_7zile',
            'nr_companii', 'volatilitate_7zile', 'hour_sin',
            'hour_cos']

df_egll = df_egll_full[features].fillna(0)
print(f'Valori lipsa: {df_egll.isnull().sum().sum()}')

WINDOW_SIZE = 24

# Split cronologic pe datele brute, inainte de scalare
split_raw = int(len(df_egll) * 0.8)

train_raw = df_egll.iloc[:split_raw]
test_raw = df_egll.iloc[split_raw - WINDOW_SIZE:]

scaler = MinMaxScaler()
scaler.fit(train_raw)

train_scaled = scaler.transform(train_raw)
test_scaled = scaler.transform(test_raw)

# Pastram date+ora corespunzatoare exact segmentului de test
test_labels = df_egll_full[['date',
```

```
'arrival_hour']].iloc[split_row:].reset_index(drop=True)

print(f'Date scalate - train: shape={train_scaled.shape}, test:
shape={test_scaled.shape}')
```

Inregistrari EGLL: 2191

Valori lipsa: 0

Date scalate - train: shape=(1752, 7), test: shape=(463, 7)

6.5 Construirea ferestrelor glisante

Creăm secvențe de antrenare și de test separat, din segmentele deja scalate independent (train_scaled, test_scaled), pentru fiecare moment t, fereastra de 24 de ore anterioare devine vectorul de input X, iar valoarea flights_count la momentul t devine eticheta y.

```
def create_sequences(data, window_size):
    X, y = [], []
    for i in range(window_size, len(data)):
        X.append(data[i-window_size:i])      # ultimele 24 de ore
        y.append(data[i, 0])                 # flights_count la
momentul t
    return np.array(X), np.array(y)

X_train, y_train = create_sequences(train_scaled, WINDOW_SIZE)
X_test, y_test = create_sequences(test_scaled, WINDOW_SIZE)

print(f'Dimensiune X_train: {X_train.shape}, X_test: {X_test.shape}')
print(f'Fiecare secventa: {WINDOW_SIZE} ore x {len(features)}
features')
```

Dimensiune X_train: (1728, 24, 7), X_test: (439, 24, 7)

Fiecare secventa: 24 ore x 7 features

6.6 Construirea și antrenarea modelului LSTM

Arhitectura rețelei constă din:

- **LSTM(64)** cu `return_sequences=True` - primul strat recurent care procesează secvența și returnează stările intermediare
- **Dropout(0.2)** - regularizare pentru prevenirea overfitting-ului
- **LSTM(32)** - al doilea strat recurent care condensează informația
- **Dropout(0.2)** - regularizare suplimentară
- **Dense(1)** - stratul de ieșire care produce predicția scalară

Folosim optimizatorul **Adam** cu funcția de pierdere **MSE** (Mean Squared Error), standard pentru regresie.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
```

```

from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# arhitectura LSTM
model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(WINDOW_SIZE,
len(features))),
    Dropout(0.2),
    LSTM(32),
    Dropout(0.2),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')
model.summary()

# EarlyStopping – opreste antrenarea daca validarea nu mai scade
early_stop = EarlyStopping(monitor='val_loss', patience=10,
restore_best_weights=True)

print('\nIncep antrenarea LSTM...')
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)
print('Antrenare finalizata!')

```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

C:\Users\Lenovo\anaconda3\envs\spark_env\Lib\site-packages\keras\src\layers\rnn\rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(**kwargs)

Model: "sequential"

Layer (type) Param #	Output Shape
lstm (LSTM)	(None, 24, 64)

Layer	Size	Dropout	Activation
input	18,432	0	None
dropout (Dropout)	(None, 24, 64)	0	None
lstm_1 (LSTM)	(None, 32)	0	None
dropout_1 (Dropout)	(None, 32)	0	None
dense (Dense)	(None, 1)	0	None

Total params: 30,881 (120.63 KB)

Trainable params: 30,881 (120.63 KB)

```
Non-trainable params: 0 (0.00 B)
```

Incep antrenarea LSTM...

Epoch 1/50

```
44/44 ————— 4s 31ms/step - loss: 0.0362 - val_loss: 0.0236
```

Epoch 2/50

```
44/44 ————— 2s 23ms/step - loss: 0.0223 - val_loss: 0.0185
```

Epoch 3/50

```
44/44 ————— 1s 23ms/step - loss: 0.0142 - val_loss: 0.0085
```

Epoch 4/50

```
44/44 ————— 1s 21ms/step - loss: 0.0139 - val_loss: 0.0093
```

Epoch 5/50

```
44/44 ————— 1s 23ms/step - loss: 0.0114 - val_loss: 0.0103
```

Epoch 6/50

```
44/44 ————— 1s 23ms/step - loss: 0.0101 - val_loss: 0.0075
```

Epoch 7/50

```
44/44 ————— 1s 23ms/step - loss: 0.0109 - val_loss: 0.0087
```

Epoch 8/50

```
44/44 ————— 1s 23ms/step - loss: 0.0103 - val loss:
```

```
0.0072
Epoch 9/50
44/44 _____ 1s 20ms/step - loss: 0.0094 - val_loss:
0.0072
Epoch 10/50
44/44 _____ 1s 22ms/step - loss: 0.0092 - val_loss:
0.0074
Epoch 11/50
44/44 _____ 1s 23ms/step - loss: 0.0093 - val_loss:
0.0081
Epoch 12/50
44/44 _____ 1s 22ms/step - loss: 0.0095 - val_loss:
0.0073
Epoch 13/50
44/44 _____ 1s 23ms/step - loss: 0.0092 - val_loss:
0.0068
Epoch 14/50
44/44 _____ 1s 21ms/step - loss: 0.0085 - val_loss:
0.0069
Epoch 15/50
44/44 _____ 1s 21ms/step - loss: 0.0089 - val_loss:
0.0084
Epoch 16/50
44/44 _____ 1s 21ms/step - loss: 0.0084 - val_loss:
0.0067
Epoch 17/50
44/44 _____ 1s 22ms/step - loss: 0.0085 - val_loss:
0.0065
Epoch 18/50
44/44 _____ 1s 20ms/step - loss: 0.0084 - val_loss:
0.0068
Epoch 19/50
44/44 _____ 1s 22ms/step - loss: 0.0075 - val_loss:
0.0063
Epoch 20/50
44/44 _____ 1s 21ms/step - loss: 0.0077 - val_loss:
0.0086
Epoch 21/50
44/44 _____ 1s 22ms/step - loss: 0.0081 - val_loss:
0.0066
Epoch 22/50
44/44 _____ 1s 21ms/step - loss: 0.0069 - val_loss:
0.0062
Epoch 23/50
44/44 _____ 1s 15ms/step - loss: 0.0072 - val_loss:
0.0067
Epoch 24/50
44/44 _____ 2s 20ms/step - loss: 0.0072 - val_loss:
0.0069
Epoch 25/50
```



```
44/44 _____ 1s 20ms/step - loss: 0.0066 - val_loss: 0.0063
Epoch 26/50
44/44 _____ 1s 21ms/step - loss: 0.0068 - val_loss: 0.0060
Epoch 27/50
44/44 _____ 1s 19ms/step - loss: 0.0062 - val_loss: 0.0051
Epoch 28/50
44/44 _____ 1s 20ms/step - loss: 0.0056 - val_loss: 0.0066
Epoch 29/50
44/44 _____ 1s 22ms/step - loss: 0.0055 - val_loss: 0.0051
Epoch 30/50
44/44 _____ 1s 21ms/step - loss: 0.0053 - val_loss: 0.0050
Epoch 31/50
44/44 _____ 1s 23ms/step - loss: 0.0055 - val_loss: 0.0052
Epoch 32/50
44/44 _____ 1s 22ms/step - loss: 0.0050 - val_loss: 0.0048
Epoch 33/50
44/44 _____ 1s 20ms/step - loss: 0.0052 - val_loss: 0.0047
Epoch 34/50
44/44 _____ 1s 23ms/step - loss: 0.0050 - val_loss: 0.0050
Epoch 35/50
44/44 _____ 1s 23ms/step - loss: 0.0048 - val_loss: 0.0049
Epoch 36/50
44/44 _____ 1s 20ms/step - loss: 0.0048 - val_loss: 0.0048
Epoch 37/50
44/44 _____ 1s 25ms/step - loss: 0.0049 - val_loss: 0.0049
Epoch 38/50
44/44 _____ 1s 22ms/step - loss: 0.0049 - val_loss: 0.0047
Epoch 39/50
44/44 _____ 1s 23ms/step - loss: 0.0048 - val_loss: 0.0050
Epoch 40/50
44/44 _____ 1s 18ms/step - loss: 0.0048 - val_loss: 0.0054
Epoch 41/50
44/44 _____ 2s 24ms/step - loss: 0.0044 - val_loss: 0.0045
```

```

Epoch 42/50
44/44 _____ 1s 22ms/step - loss: 0.0047 - val_loss: 0.0052
Epoch 43/50
44/44 _____ 1s 19ms/step - loss: 0.0051 - val_loss: 0.0047
Epoch 44/50
44/44 _____ 1s 22ms/step - loss: 0.0049 - val_loss: 0.0052
Epoch 45/50
44/44 _____ 1s 18ms/step - loss: 0.0048 - val_loss: 0.0047
Epoch 46/50
44/44 _____ 1s 21ms/step - loss: 0.0047 - val_loss: 0.0048
Epoch 47/50
44/44 _____ 1s 24ms/step - loss: 0.0049 - val_loss: 0.0046
Epoch 48/50
44/44 _____ 1s 21ms/step - loss: 0.0048 - val_loss: 0.0063
Epoch 49/50
44/44 _____ 1s 22ms/step - loss: 0.0048 - val_loss: 0.0044
Epoch 50/50
44/44 _____ 1s 22ms/step - loss: 0.0046 - val_loss: 0.0046
Antrenare finalizata!

```

6.7 Evaluarea modelului LSTM

Evaluăm modelul pe setul de test și vizualizăm predicțiile față de valorile reale. Reconvertim predicțiile la scara originală (număr de zboruri) folosind inversa transformării MinMaxScaler pentru a putea interpreta erorile în unități reale.

```

import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

predictions_scaled = model.predict(X_test)

def inverse_scale(predictions, y, features_len):
    placeholder = np.zeros((len(predictions), features_len))
    placeholder[:, 0] = predictions.flatten()
    return scaler.inverse_transform(placeholder)[:, 0]

predictions_real = inverse_scale(predictions_scaled, y_test,
len(features))
y_test_real = inverse_scale(y_test.reshape(-1, 1), y_test,

```

```

len(features))

# "data ora:00"
n_points = 100
axis_labels = [f"{row.date} {int(row.arrival_hour):02d}:00" for row in
test_labels.iloc[:n_points].itertuples())

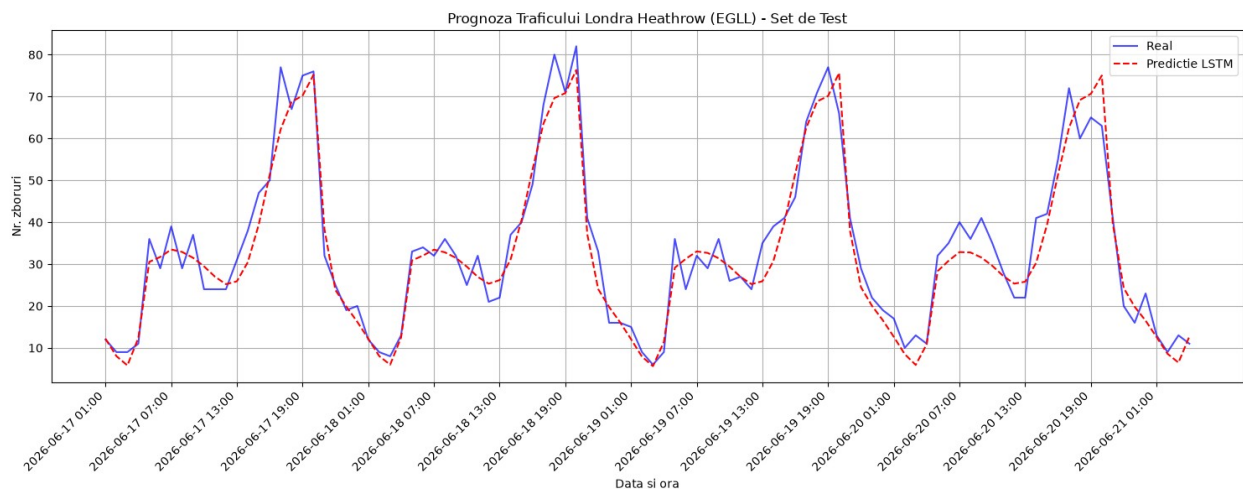
plt.figure(figsize=(15, 6))
plt.plot(y_test_real[:n_points], label='Real', color='blue',
alpha=0.7)
plt.plot(predictions_real[:n_points], label='Predictie LSTM',
color='red', linestyle='--')
plt.title('Proгноза Traficului Londra Heathrow (EGLL) - Set de Test')
plt.xlabel('Data si ora')
plt.ylabel('Nr. zboruri')

# o eticheta la fiecare 6 puncte, ca sa nu se suprapuna
step = 6
plt.xticks(ticks=range(0, n_points, step), labels=axis_labels[::step],
rotation=45, ha='right')

plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

14/14 ————— 1s 28ms/step

```



```

from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

mae = mean_absolute_error(y_test_real, predictions_real)
rmse = np.sqrt(mean_squared_error(y_test_real, predictions_real))
r2 = r2_score(y_test_real, predictions_real)

```

```

print(f'=== Rezultate LSTM ===')
print(f'MAE:  {mae:.2f} zboruri/ora')
print(f'RMSE: {rmse:.2f} zboruri/ora')
print(f'R2:   {r2:.2f}')
print('-----')

df_results = pd.DataFrame({
    'Data': test_labels['date'].iloc[-10:].values,
    'Ora': test_labels['arrival_hour'].iloc[-10:].values,
    'Real': y_test_real[-10:],
    'Predictie': predictions_real[-10:],
    'Eroare': np.abs(y_test_real[-10:] - predictions_real[-10:])
})
print('\nPredictii comparate (ultimele 10 - cele mai recente):')
print(df_results)

```

```

=== Rezultate LSTM ===
MAE:  6.19 zboruri/ora
RMSE: 9.23 zboruri/ora
R2:   0.81
-----

```

Predictii comparate (ultimele 10 - cele mai recente):

	Data	Ora	Real	Predictie	Eroare
0	2026-07-05	9	2.0	14.449036	12.449036
1	2026-07-05	10	3.0	27.505524	24.505524
2	2026-07-05	12	1.0	30.518558	29.518558
3	2026-07-05	13	4.0	32.406510	28.406510
4	2026-07-05	14	1.0	32.400555	31.400555
5	2026-07-05	16	1.0	32.746654	31.746654
6	2026-07-05	19	1.0	34.749439	33.749439
7	2026-07-05	20	4.0	38.725338	34.725338
8	2026-07-05	21	17.0	43.715241	26.715241
9	2026-07-05	22	3.0	49.083828	46.083828

6.8 Importanța feature-urilor pentru LSTM

Spre deosebire de RandomForest, LSTM nu oferă importanța feature-urilor direct. Folosim **permutation importance**: permutăm aleator valorile unui feature și măsurăm cât crește eroarea - un feature important va cauza o creștere mare a erorii când este perturbat.

```

def get_feature_importance(model, X_test, y_test, feature_names):
    baseline_pred = model.predict(X_test, verbose=0).flatten()
    baseline_error = mean_squared_error(y_test, baseline_pred)
    importances = []
    for i in range(X_test.shape[2]):
        X_permuted = X_test.copy()
        np.random.shuffle(X_permuted[:, :, i])
        permuted_pred = model.predict(X_permuted, verbose=0).flatten()

```

```

        permuted_error = mean_squared_error(y_test, permuted_pred)
        importances.append(permuted_error - baseline_error)
    return np.array(importances)

importances = get_feature_importance(model, X_test, y_test, features)
importances = np.maximum(importances, 0)
importances_percent = (importances / np.sum(importances)) * 100

df_importance = pd.DataFrame({
    'Feature': features,
    'Importanta (%)': importances_percent
}).sort_values(by='Importanta (%)', ascending=False)

print('=== Importanta Feature-urilor (Permutation Importance) ===')
print(df_importance.to_string(index=False, formatters={'Importanta (%)': '{:.2f}%'.format}))

=== Importanta Feature-urilor (Permutation Importance) ===
      Feature Importanta (%)
    hour_sin           35.03%
   media_7zile          34.47%
    hour_cos           13.20%
 flights_yesterday      8.74%
   flights_count         8.08%
    nr_companii          0.49%
 volatilitate_7zile      0.00%

```

Observații

1. **RandomForest** obține un scor bun ($R^2 \approx 0.84$), apropiat de cel al **LSTM** ($R^2 \approx 0.81-0.83$) — diferența dintre cele două metode este mică, ambele exploatând eficient feature-uri de tip lag (`media_7zile`, `flights_yesterday`) care captează pattern-urile repetitive ale traficului aerian.
2. **LSTM** rămâne util pentru contextul secvențial complet (ex. detectarea tendințelor de creștere/scădere pe parcursul zilei)
3. **KMeans** identifică corect că LROP este un aeroport regional cu volum semnificativ mai mic față de hub-urile vest-europene (EGLL, LFPG, EDDF, EHAM), iar corelația predicție-realitate a RandomForest confirmă acest tipar: LROP are cea mai slabă corelație (~ 0.83), în timp ce EGLL, cel mai mare hub, are cea mai bună (~ 0.92).
4. Feature-ul `volatilitate_7zile` a avut, în mod neașteptat, importanță aproape nulă atât pentru RandomForest cât și pentru LSTM — traficul, deși variază de la o zi la alta, nu pare să aibă o componentă de "impredictibilitate" separabilă de nivelul mediu istoric al fiecărei ore, semnal deja captat mai eficient de `media_7zile` și `hour_sin/hour_cos`.

```

spark.stop()
print('SparkSession oprit. Notebook 2 finalizat!')

```

Notebook 3 - Proces de Streaming în Timp Real

Descriere generală

Acest notebook implementează un **sistem de monitorizare în timp real** a traficului aerian deasupra celor 5 aeroporturi europene monitorizate în proiect, folosind **Apache Kafka** și **Spark Structured Streaming**.

Arhitectura sistemului

```
OpenSky Network API
  ↓ (get_states() la fiecare 60 secunde)
Kafka Producător → Topic: 'zboruri'
  ↓
Spark Structured Streaming
  ↓ (foreachBatch la fiecare 60 secunde)
Procesare + Clasificare + Afișare raport
```

Componentele sistemului

Componentă	Tehnologie	Rol
Producer	Python + OpenSky API	Colectează pozițiile live ale avioanelor și le publică în Kafka
Broker	Apache Kafka	Sistemul de mesagerie care decuplează producătorul de consumator
Consumer	Spark Structured Streaming	Citește mesajele din Kafka, le procesează și generează rapoarte

7. Proces de streaming

7.1. Inițializarea SparkSession cu suport Kafka

Pornim o sesiune Spark cu pachetul `spark-sql-kafka` care permite citirea din topicuri Kafka ca sursă de streaming. Configurăm și variabilele Hadoop necesare pe Windows pentru a evita erorile legate de permisiuni de fișiere.

```
import os
os.environ["HADOOP_HOME"] = r"C:\hadoop"
os.environ["PATH"] += r";C:\hadoop\bin"

from pyspark.sql import SparkSession

spark = SparkSession.builder \
```

```

    .appName("StreamingFlights") \
    .master("local[*]") \
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-
0-10_2.12:3.5.3") \
    .getOrCreate()

print(f"Spark version: {spark.version}")
print("Sesiune pornită cu succes!")

Spark version: 3.5.3
Sesiune pornită cu succes!

```

7.2. Definirea schemei și a zonelor geografice monitorizate

7.2.1 Schema mesajelor Kafka

Kafka transmite mesajele ca bytes bruti. Definim schema JSON a mesajelor produse de OpenSky API, astfel încât Spark să știe cum să deserializeze fiecare câmp. Câmpurile provin direct din răspunsul `get_states()` al OpenSky Network:

Câmp	Tip	Descriere
<code>icao24</code>	String	Adresa unică ICAO a transponderului
<code>callsign</code>	String	Indicativul zborului (ex: ROT384J)
<code>origin_country</code>	String	Țara de origine inferată din ICAO24
<code>latitude, longitude</code>	Float	Coordonatele WGS-84 ale aeronavei
<code>baro_altitude</code>	Float	Altitudinea barometrică în metri
<code>velocity</code>	Float	Viteza față de sol în m/s
<code>vertical_rate</code>	Float	Rata de urcare/coborâre în m/s (pozitiv = urcă)
<code>on_ground</code>	Boolean	True dacă aeronava e pe pistă
<code>category</code>	Integer	Categoria aeronavei (0=necunoscut, 4=mare, 6=heavy)

7.2.2 Zonele geografice ale aeroporturilor

Definim bounding box-urile în jurul fiecărui aeroport monitorizat. O aeronavă este considerată în zona unui aeroport dacă coordonatele sale se află în interiorul bounding box-ului corespunzător.

```

from pyspark.sql.functions import col, from_json, when, round as
spark_round

```

```

from pyspark.sql.types import (
    StructType, StringType, FloatType,
    LongType, BooleanType, IntegerType
)
from datetime import datetime

# Schema mesajelor JSON primite din Kafka
schema = StructType() \
    .add("timestamp", LongType()) \
    .add("icao24", StringType()) \
    .add("callsign", StringType()) \
    .add("origin_country", StringType()) \
    .add("latitude", FloatType()) \
    .add("longitude", FloatType()) \
    .add("baro_altitude", FloatType()) \
    .add("velocity", FloatType()) \
    .add("vertical_rate", FloatType()) \
    .add("on_ground", BooleanType()) \
    .add("category", IntegerType())

# Bounding box-urile geografice ale aeroporturilor monitorizate
# (lat_min, lat_max) si (lon_min, lon_max)
AIRPORTS = {
    "EDDF": {"lat": (49.90, 50.20), "lon": (8.40, 8.80), "name":
"Frankfurt"},
    "EGLL": {"lat": (51.40, 51.55), "lon": (-0.55, -0.35), "name":
"Heathrow"},
    "LFPG": {"lat": (48.95, 49.08), "lon": (2.45, 2.65), "name":
"Paris CDG"},
    "EHAM": {"lat": (52.25, 52.40), "lon": (4.70, 4.85), "name":
"Amsterdam"},
    "LROP": {"lat": (44.50, 44.65), "lon": (26.00, 26.20), "name":
"Bucuresti"},
}

print(f"Schema definita cu {len(schema.fields)} campuri")
print(f"Aeroporturi monitorizate: {'', ' '.join(AIRPORTS.keys())}")

```

Schema definita cu 11 campuri
Aeroporturi monitorizate: EDDF, EGLL, LFPG, EHAM, LROP

7.3. Citirea din Kafka și procesarea streamului

7.3.1 Conectarea la topicul Kafka

Spark Structured Streaming citește continuu mesajele din topicul `zboruri` de pe brokerul Kafka local. Fiecare mesaj conține poziția unui avion la un moment dat, serializată ca JSON.


```
df_kafka = (
    spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", "localhost:9092")
        .option("subscribe", "zboruri")
        .load()
)

print("Stream Kafka conectat la topicul 'zboruri'")
print(f"Schema raw Kafka: {df_kafka.schema.simpleString()}")

Stream Kafka conectat la topicul 'zboruri'
Schema raw Kafka:
struct<key:binary,value:binary,topic:string,partition:int,offset:bigint,timestamp:timestamp,timestampType:int>
```

7.3.2 Parsarea JSON și clasificarea aeronavelor

Mesajele Kafka sosesc ca bytes bruti în coloana `value`. Aplicăm trei transformări:

1. **Deserializare JSON** - convertim bytes → string → structură cu `from_json()`
2. **Conversie viteză** - transformăm din m/s în km/h
3. **Clasificare status zbor** - pe baza `baro_altitude` și `on_ground` clasificăm fiecare aeronavă:
 - **LA SOL** - `on_ground = True`
 - **DECOLARE/ATERIZARE** - sub 1000m altitudine
 - **APROPIERE** - între 1000m și 3000m (faza finală de apropiere)
 - **SURVOLARE** - peste 3000m (zbor de croazieră sau tranzit)
4. **Clasificare direcție** - pe baza `vertical_rate` determinăm dacă aeronava urcă, coboară sau menține altitudinea

```
raw_df = (
    df_kafka
        .selectExpr("CAST(value AS STRING)")
        .select(from_json(col("value"), schema).alias("data"))
        .select("data.*")
        .withColumn("velocity_kmh", spark_round(col("velocity") * 3.6,
1))
    # Clasificare status zbor pe baza altitudinii si starii la sol
    .withColumn("flight_status",
        when(col("on_ground") == True, "LA SOL")
        .when(col("baro_altitude") < 1000, "DECOLARE/ATERIZARE")
        .when(col("baro_altitude") < 3000, "APROPIERE")
        .otherwise("SURVOLARE"))
    # Clasificare directie pe baza ratei verticale
    .withColumn("directie",
        when(col("vertical_rate") > 1, "URCARE")
        .when(col("vertical_rate") < -1, "COBORARE")
```

```

        .otherwise("NIVEL"))
    )

print("Au fost aplicate o serie de transformari asupra streamului")
Au fost aplicate o serie de transformari asupra streamului

```

7.4. Funcția de procesare per batch (foreachBatch)

Spark Structured Streaming procesează datele în **micro-batch-uri** - la fiecare 60 de secunde colectează toate mesajele noi din Kafka și le trimite funcției `process_batch`.

Funcția primește un DataFrame Spark cu toate aeronavele detectate în intervalul respectiv și:

1. Îl convertește în Pandas pentru procesare rapidă
2. Filtrează aeronavele per aeroport folosind bounding box-urile geografice
3. Calculează statistici: total detectate, în aer vs la sol, viteză și altitudine medie
4. Afișează un raport structurat per aeroport și un sumar global

```

def process_batch(batch_df, epoch_id):
    """
    Primește un DataFrame cu toate aeronavele detectate în intervalul
    respectiv.
    """
    df_p = batch_df.toPandas()
    ora = datetime.now().strftime('%H:%M:%S')

    print(f"\n{'='*70}")
    print(f"  MONITORIZARE TRAFIC AERIAN   |   Batch {epoch_id}   |
{ora}")
    print(f"{'='*70}")

    if df_p.empty:
        print("  Nicio aeronavă detectată.")
        print(f"{'='*70}")
        return

    total_detectate = 0

    # fiecare aeroport in parte
    for icao, info in AIRPORTS.items():
        zona = df_p[
            df_p['latitude'].between(*info["lat"]) &
            df_p['longitude'].between(*info["lon"])
        ]

        if zona.empty:
            print(f"\n  {info['name']} ({icao})")
            print(f"{'-'*50}")
            print("  Nicio aeronavă detectată.")

```

```

total_detectate += len(zona)
in_aer = zona[zona['on_ground'] == False]
la_sol = zona[zona['on_ground'] == True]
aproape = zona[zona['baro_altitude'] < 3000]

print(f"\n {info['name']} ({icao})")
print(f" {'-'*50}")
print(f" Total : {len(zona)} | In aer: {len(in_aer)} | La
sol: {len(la_sol)}")

if not in_aer.empty:
    print(f" Viteza medie :
{in_aer['velocity_kmh'].mean():.1f} km/h")
    print(f" Altitudine medie:
{in_aer['baro_altitude'].mean():.0f} m")

if not aproape.empty:
    print(f" In apropiere/aterizare: {len(aproape)}
aeronave")

# tabel cu aeronavele detectate
print(f"\n Aeronave detectate:")
cols = ['callsign', 'origin_country', 'baro_altitude',
        'velocity_kmh', 'flight_status', 'directie']
print(zona[cols].to_string(index=False))

# top tari de origine
print(f"\n Top tari origine:")
for tara, cnt in
zona['origin_country'].value_counts().head(5).items():
    print(f" {tara:<20} {cnt}")

# Sumar
print(f"\n{'-'*70}")
print(f" Total aeronave procesate in acest batch:
{total_detectate}")
print(f"{'='*70}")

```

7.5. Pornirea streaming-ului

Pornim query-ul de streaming cu următoarea configurație:

- **outputMode("append")** - fiecare batch adaugă noi date (nu suprascrie)
- **foreachBatch(process_batch)** - la fiecare batch se apelează funcția noastră de procesare
- **trigger(processingTime='60 seconds')** - Spark colectează și procesează mesajele la fiecare 60 de secunde
- **awaitTermination()** - păstrează notebook-ul activ până la oprirea manuală

```

query = (
    raw_df.writeStream
        .outputMode("append")
        .foreachBatch(process_batch)
        .trigger(processingTime='60 seconds')
        .start()
)

print("Procesare la fiecare 60 secunde.")
query.awaitTermination()

```

Procesare la fiecare 60 secunde.

```

=====
MONITORIZARE TRAFIC AERIAN | Batch 0 | 17:55:01
=====
Nicio aeronavă detectată.
=====

```

```

=====
MONITORIZARE TRAFIC AERIAN | Batch 1 | 17:56:03
=====

```

Frankfurt (EDDF)

Total : 19 | In aer: 10 | La sol: 9
 Viteza medie : 517.6 km/h
 Altitudine medie: 5155 m
 In apropiere/aterizare: 5 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
flight_status directie			
LC01511	United States	NaN	46.3
LA SOL NIVEL			
DHXCG	Germany	129.539993	9.3
DECOLARE/ATERIZARE COBORARE			
DLA8MM	Italy	NaN	25.9
LA SOL NIVEL			
RAM810D	Morocco	NaN	21.3
LA SOL NIVEL			
BGA131G	France	10058.400391	805.6
SURVOLARE NIVEL			
ANA223	Japan	259.079987	264.9
DECOLARE/ATERIZARE COBORARE			
DLH6VX	Germany	1485.900024	419.8
APROPIERE URCARE			
DLH71X	Germany	NaN	48.2
LA SOL NIVEL			
DLH4PK	Germany	10370.820312	781.5

SURVOLARE	NIVEL			
DLH830		Germany	624.840027	332.0
DECOLARE/ATERIZARE	URCARE			
DLH1LA		Germany	NaN	18.5
LA SOL	NIVEL			
DLH35M		Germany	NaN	7.4
LA SOL	NIVEL			
BAW865		United Kingdom	10972.799805	757.7
SURVOLARE	NIVEL			
DLH9YW		Germany	NaN	13.0
LA SOL	NIVEL			
DLH2TT		Germany	6606.540039	720.8
SURVOLARE	COBORARE			
DEQFE		Germany	373.380005	195.7
DECOLARE/ATERIZARE	COBORARE			
KLM37J		Kingdom of the Netherlands	10668.000000	888.6
SURVOLARE	NIVEL			
DLA7MT		Italy	NaN	0.0
LA SOL	NIVEL			
OCN6GP		Germany	NaN	27.8
LA SOL	NIVEL			

Top tari origine:

Germany	11
Italy	2
United States	1
Morocco	1
France	1

Heathrow (EGLL)

Total : 6 | In aer: 4 | La sol: 2
Viteza medie : 254.6 km/h
Altitudine medie: 133 m
In apropiere/aterizare: 4 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	
flight_status	directie			
SIA308	Singapore	NaN	17.6	LA
SOL	NIVEL			
BAW19N	United Kingdom	327.660004	314.9	
DECOLARE/ATERIZARE	URCARE			
BAW4LV	United Kingdom	38.099998	242.6	
DECOLARE/ATERIZARE	COBORARE			
BAW91J	United Kingdom	30.480000	238.9	
DECOLARE/ATERIZARE	COBORARE			
BAW749	United Kingdom	137.160004	222.2	
DECOLARE/ATERIZARE	COBORARE			
EIN168	Ireland	NaN	42.6	LA

SOL NIVEL

Top tari origine:

United Kingdom	4
Singapore	1
Ireland	1

Paris CDG (LFPG)

Total : 5 | In aer: 5 | La sol: 0

Viteza medie : 252.0 km/h

Altitudine medie: 215 m

In apropiere/aterizare: 5 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
AFR1623	France	121.919998	239.8
DECOLARE/ATERIZARE	COBORARE		
DAH1528	Algeria	167.639999	245.3
DECOLARE/ATERIZARE	COBORARE		
MAU15	Mauritius	495.299988	314.1
DECOLARE/ATERIZARE	URCARE		
AFR23TC	France	137.160004	226.7
DECOLARE/ATERIZARE	COBORARE		
AFR78WT	France	152.399994	234.0
DECOLARE/ATERIZARE	COBORARE		

Top tari origine:

France	3
Algeria	1
Mauritius	1

Amsterdam (EHAM)

Total : 15 | In aer: 5 | La sol: 10

Viteza medie : 458.7 km/h

Altitudine medie: 4581 m

In apropiere/aterizare: 3 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
EXS36PN	United Kingdom	11582.400391	734.9
SURVOLARE	NIVEL		
N806CH	United States	NaN	21.3
LA SOL	NIVEL		
NaN	Kingdom of the Netherlands	NaN	37.0
LA SOL	NIVEL		
RAM850W	Morocco	NaN	40.8

LA SOL	NIVEL			
KLM36E	Kingdom of the Netherlands	-30.480000		148.4
DECOLARE/ATERIZARE	NIVEL			
DLH6JF	Germany	NaN		19.4
LA SOL	NIVEL			
TFL6DY	Belgium	NaN		29.6
LA SOL	NIVEL			
EIN60G	Ireland	NaN		50.0
LA SOL	NIVEL			
CND2J	Kingdom of the Netherlands	NaN		26.9
LA SOL	NIVEL			
KLM898	Kingdom of the Netherlands	NaN		9.3
LA SOL	NIVEL			
AWH26T	Germany	11277.599609		881.4
SURVOLARE	NIVEL			
TRA5791	Kingdom of the Netherlands	NaN		29.6
LA SOL	NIVEL			
KLM24V	France	99.059998		244.8
DECOLARE/ATERIZARE	COBORARE			
NaN	Saudi Arabia	-22.860001		283.9
DECOLARE/ATERIZARE	NIVEL			
NUM221	Switzerland	NaN		63.0
LA SOL	NIVEL			

Top tari origine:

Kingdom of the Netherlands	5
Germany	2
United Kingdom	1
United States	1
Morocco	1

Bucuresti (LROP)

Total : 1 | In aer: 1 | La sol: 0
Viteza medie : 183.9 km/h
Altitudine medie: 1356 m
In apropiere/aterizare: 1 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	flight_status
directie				
CROSS51	United States	1356.359985	183.9	APROPIERE
URCARE				

Top tari origine:

United States	1
---------------	---

Total aeronave procesate in acest batch: 46

=====

MONITORIZARE TRAFIC AERIAN | Batch 2 | 17:57:01

=====

Frankfurt (EDDF)

Total : 15 | In aer: 6 | La sol: 9

Viteza medie : 309.8 km/h

Altitudine medie: 2413 m

In apropiere/aterizare: 5 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	
LC01511	United States	NaN	51.8	LA
SOL	NIVEL			
DEREA	Germany	2164.080078	230.9	
APROPIERE	URCARE			
DHXCG	Germany	129.539993	9.3	
DECOLARE/ATERIZARE	COBORARE			
DLA8MM	Italy	NaN	25.9	LA
SOL	NIVEL			
RAM810D	Morocco	NaN	15.7	LA
SOL	NIVEL			
ANA223	Japan	76.199997	250.8	
DECOLARE/ATERIZARE	NIVEL			
DLH71X	Germany	NaN	48.2	LA
SOL	NIVEL			
DLH4PK	Germany	10363.200195	773.2	
SURVOLARE	NIVEL			
DLH830	Germany	1341.119995	402.0	
APROPIERE	URCARE			
NaN	Germany	NaN	38.9	LA
SOL	NIVEL			
DLH35M	Germany	NaN	7.4	LA
SOL	NIVEL			
DLH9YW	Germany	NaN	13.0	LA
SOL	NIVEL			
DEQFE	Germany	403.859985	192.5	
DECOLARE/ATERIZARE	URCARE			
DLA7MT	Italy	NaN	0.0	LA
SOL	NIVEL			
OCN6GP	Germany	NaN	27.8	LA
SOL	NIVEL			

Top tari origine:

Germany 10

Italy 2

United States 1

Morocco	1
Japan	1

Heathrow (EGLL)

Total : 5 | In aer: 4 | La sol: 1
Viteza medie : 341.7 km/h
Altitudine medie: 2006 m
In apropiere/aterizare: 3 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	
flight_status	directie			
SIA308	Singapore	NaN	26.9	LA
SOL	NIVEL			
EZY19FU	United Kingdom	7924.799805	663.0	
SURVOLARE	NIVEL			
BAW4LV	United Kingdom	38.099998	242.6	
DECOLARE/ATERIZARE	COBORARE			
BAW91J	United Kingdom	30.480000	238.9	
DECOLARE/ATERIZARE	COBORARE			
BAW749	United Kingdom	30.480000	222.3	
DECOLARE/ATERIZARE	COBORARE			

Top tari origine:

United Kingdom	4
Singapore	1

Paris CDG (LFPG)

Total : 4 | In aer: 4 | La sol: 0
Viteza medie : 262.1 km/h
Altitudine medie: 322 m
In apropiere/aterizare: 4 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	
flight_status	directie			
AFR1623	France	121.919998	239.8	
DECOLARE/ATERIZARE	COBORARE			
DAH1528	Algeria	30.480000	249.0	
DECOLARE/ATERIZARE	COBORARE			
MAU15	Mauritius	982.979980	325.6	
DECOLARE/ATERIZARE	URCARE			
AFR78WT	France	152.399994	234.0	
DECOLARE/ATERIZARE	COBORARE			

Top tari origine:

France	2
Algeria	1

Mauritius 1

Amsterdam (EHAM)

Total : 14 | In aer: 2 | La sol: 12

Viteza medie : 274.8 km/h

Altitudine medie: 290 m

In apropiere/aterizare: 2 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
flight_status directie			
KLM617	Kingdom of the Netherlands	NaN	35.2
LA SOL NIVEL			
N806CH	United States	NaN	21.3
LA SOL NIVEL			
KLM79J	Kingdom of the Netherlands	NaN	37.0
LA SOL NIVEL			
RAM850W	Morocco	NaN	55.5
LA SOL NIVEL			
KLM36E	Kingdom of the Netherlands	NaN	55.5
LA SOL NIVEL			
DLH6JF	Germany	NaN	37.0
LA SOL NIVEL			
TFL6DY	Belgium	NaN	29.6
LA SOL NIVEL			
EIN60G	Ireland	NaN	31.5
LA SOL NIVEL			
CND2J	Kingdom of the Netherlands	NaN	29.6
LA SOL NIVEL			
KLM898	Kingdom of the Netherlands	NaN	9.3
LA SOL NIVEL			
TRA5791	Kingdom of the Netherlands	NaN	29.6
LA SOL NIVEL			
KLM24V	France	-30.480000	172.5
DECOLARE/ATERIZARE	NIVEL		
SVA214	Saudi Arabia	609.599976	377.0
DECOLARE/ATERIZARE	URCARE		
NUM221	Switzerland	NaN	8.3
LA SOL NIVEL			

Top tari origine:

Kingdom of the Netherlands 6

United States 1

Morocco 1

Germany 1

Belgium 1

Bucuresti (LROP)

Total : 1 | In aer: 1 | La sol: 0
Viteza medie : 183.9 km/h
Altitudine medie: 1356 m
In apropiere/aterizare: 1 aeronave

Aeronave detectate:
callsign origin_country baro_altitude velocity_kmh flight_status
directie
CR0551 United States 1356.359985 183.9 APROPIERE
URCARE

Top tari origine:
United States 1

Total aeronave procesate in acest batch: 39

MONITORIZARE TRAFIC AERIAN | Batch 3 | 17:58:01

Frankfurt (EDDF)

Total : 16 | In aer: 7 | La sol: 9
Viteza medie : 240.3 km/h
Altitudine medie: 865 m
In apropiere/aterizare: 7 aeronave

Aeronave detectate:
callsign origin_country baro_altitude velocity_kmh
flight_status directie
LC01511 United States NaN 51.8 LA
SOL NIVEL
DEREA Germany 2438.399902 277.6
APROPIERE NIVEL
DHXCG Germany 129.539993 9.3
DECOLARE/ATERIZARE COBORARE
DLA8MM Italy NaN 25.9 LA
SOL NIVEL
RAM810D Morocco NaN 0.5 LA
SOL NIVEL
ANA223 Japan NaN 44.5 LA
SOL NIVEL
DLH8AU Germany 236.220001 281.3
DECOLARE/ATERIZARE URCARE
DLH71X Germany 419.100006 267.2
DECOLARE/ATERIZARE URCARE
DLH830 Germany 2026.920044 451.0
APROPIERE URCARE

SOL	NaN	Germany	NaN	27.8	LA
	NIVEL				
DLH35M		Germany	NaN	7.4	LA
SOL	NIVEL				
DLH9YW		Germany	NaN	5.5	LA
SOL	NIVEL				
DEQFE		Germany	396.239990	193.1	
DECOLARE/ATERIZARE	COBORARE				
DEWCP		Germany	411.480011	202.9	
DECOLARE/ATERIZARE	NIVEL				
DLA7MT		Italy	NaN	0.0	LA
SOL	NIVEL				
OCN6GP		Germany	NaN	27.8	LA
SOL	NIVEL				

Top tari origine:

Germany	11
Italy	2
United States	1
Morocco	1
Japan	1

Heathrow (EGLL)

Total : 5 | In aer: 4 | La sol: 1
Viteza medie : 262.6 km/h
Altitudine medie: 124 m
In apropiere/aterizare: 4 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	
flight_status	directie			
SIA308	Singapore	NaN	26.9	LA
SOL	NIVEL			
DAL37	United States	396.239990	346.6	
DECOLARE/ATERIZARE	URCARE			
BAW4LV	United Kingdom	38.099998	242.6	
DECOLARE/ATERIZARE	COBORARE			
BAW91J	United Kingdom	30.480000	238.9	
DECOLARE/ATERIZARE	COBORARE			
BAW749	United Kingdom	30.480000	222.3	
DECOLARE/ATERIZARE	COBORARE			

Top tari origine:

United Kingdom	3
Singapore	1
United States	1

Paris CDG (LFPG)

Total : 4 | In aer: 4 | La sol: 0
Viteza medie : 277.8 km/h
Altitudine medie: 330 m
In apropiere/aterizare: 4 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
AFR8F	France	960.119995	375.4
DECOLARE/ATERIZARE	URCARE		
DAH1528	Algeria	30.480000	249.0
DECOLARE/ATERIZARE	COBORARE		
EZY17LA	United Kingdom	175.259995	252.7
DECOLARE/ATERIZARE	COBORARE		
AFR78WT	France	152.399994	234.0
DECOLARE/ATERIZARE	COBORARE		

Top tari origine:

France	2
Algeria	1
United Kingdom	1

Amsterdam (EHAM)

Total : 16 | In aer: 2 | La sol: 14
Viteza medie : 181.8 km/h
Altitudine medie: 80 m
In apropiere/aterizare: 2 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
KLM71Y	Kingdom of the Netherlands	NaN	0.0
LA SOL	NIVEL		
KLM617	Kingdom of the Netherlands	NaN	37.0
LA SOL	NIVEL		
N806CH	United States	NaN	21.3
LA SOL	NIVEL		
KLM79J	Kingdom of the Netherlands	NaN	37.0
LA SOL	NIVEL		
NaN	Kingdom of the Netherlands	NaN	0.0
LA SOL	NIVEL		
BAW3KG	United Kingdom	190.50	259.7
DECOLARE/ATERIZARE	COBORARE		
RAM850W	Morocco	NaN	55.5
LA SOL	NIVEL		
KLM36E	Kingdom of the Netherlands	NaN	53.7
LA SOL	NIVEL		
DLH6JF	Germany	NaN	51.8
LA SOL	NIVEL		

TFL6DY		Belgium	NaN	29.6
LA SOL	NIVEL			
EIN60G		Ireland	NaN	31.5
LA SOL	NIVEL			
CND2J	Kingdom of the Netherlands		NaN	4.6
LA SOL	NIVEL			
KLM898	Kingdom of the Netherlands		NaN	9.3
LA SOL	NIVEL			
TRA5791	Kingdom of the Netherlands		NaN	29.6
LA SOL	NIVEL			
KLM24V		France	-30.48	104.0
DECOLARE/ATERIZARE	NIVEL			
NUM221		Switzerland	NaN	8.3
LA SOL	NIVEL			

Top tari origine:

Kingdom of the Netherlands	8
United States	1
United Kingdom	1
Morocco	1
Germany	1

Bucuresti (LROP)

Nicio aeronavă detectată.

Bucuresti (LROP)

Total : 0 | In aer: 0 | La sol: 0

Aeronave detectate:

Empty DataFrame

Columns: [callsign, origin_country, baro_altitude, velocity_kmh, flight_status, directie]

Index: []

Top tari origine:

Total aeronave procesate in acest batch: 41

MONITORIZARE TRAFIC AERIAN | Batch 4 | 17:59:01

Frankfurt (EDDF)

Total : 15 | In aer: 4 | La sol: 11
Viteza medie : 248.0 km/h

Altitudine medie: 592 m
In apropiere/aterizare: 4 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	flight_status	directie
LC01511	United States	NaN	51.8		LA
SOL	NIVEL				
DHXCG	Germany	129.539993	9.3		
DECOLARE/ATERIZARE COBORARE					
DLA8MM	Italy	NaN	38.9		LA
SOL	NIVEL				
RAM810D	Morocco	NaN	0.0		LA
SOL	NIVEL				
ANA223	Japan	NaN	25.0		LA
SOL	NIVEL				
DLH8AU	Germany	830.580017	362.7		
DECOLARE/ATERIZARE URCARE					
DLH71X	Germany	990.599976	431.7		
DECOLARE/ATERIZARE URCARE					
	NaN	Germany	NaN	22.2	LA
SOL	NIVEL				
	NaN	Germany	NaN	50.0	LA
SOL	NIVEL				
DLH35M	Germany	NaN	7.4		LA
SOL	NIVEL				
DLH88H	Germany	NaN	42.6		LA
SOL	NIVEL				
DLH9YW	Germany	NaN	259.5		LA
SOL	NIVEL				
DEWCP	Germany	419.100006	188.2		
DECOLARE/ATERIZARE NIVEL					
DLA7MT	Italy	NaN	0.0		LA
SOL	NIVEL				
OCN6GP	Germany	NaN	27.8		LA
SOL	NIVEL				

Top tari origine:

Germany	10
Italy	2
United States	1
Morocco	1
Japan	1

Heathrow (EGLL)

Total : 5 | In aer: 4 | La sol: 1
Viteza medie : 265.5 km/h
Altitudine medie: 103 m
In apropiere/aterizare: 4 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	flight_status	directie
SIA308	Singapore	NaN	26.9		LA
SOL	NIVEL				
BAW59FT	United Kingdom	152.399994	238.9		
DECOLARE/ATERIZARE COBORARE					
ACA861	Canada	198.119995	361.8		
DECOLARE/ATERIZARE URCARE					
BAW91J	United Kingdom	30.480000	238.9		
DECOLARE/ATERIZARE COBORARE					
BAW749	United Kingdom	30.480000	222.3		
DECOLARE/ATERIZARE COBORARE					

Top tari origine:

United Kingdom	3
Singapore	1
Canada	1

Paris CDG (LFPG)

Total : 5 | In aer: 5 | La sol: 0
Viteza medie : 291.4 km/h
Altitudine medie: 305 m
In apropiere/aterizare: 5 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	flight_status	directie
DILUI	Germany	701.039978	364.2		
DECOLARE/ATERIZARE COBORARE					
NaN	France	327.660004	336.5		
DECOLARE/ATERIZARE URCARE					
DAH1528	Algeria	30.480000	249.0		
DECOLARE/ATERIZARE COBORARE					
EZY17LA	United Kingdom	121.919998	250.8		
DECOLARE/ATERIZARE COBORARE					
DLH16A	Germany	342.899994	256.4		
DECOLARE/ATERIZARE COBORARE					

Top tari origine:

Germany	2
France	1
Algeria	1
United Kingdom	1

Amsterdam (EHAM)

Total : 16 | In aer: 2 | La sol: 14

Viteza medie : 341.6 km/h
Altitudine medie: 225 m
In apropiere/aterizare: 2 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
flight_status directie			
KLM71Y	Kingdom of the Netherlands	NaN	0.0
LA SOL NIVEL			
KLM617	Kingdom of the Netherlands	NaN	37.0
LA SOL NIVEL			
N806CH	United States	NaN	21.3
LA SOL NIVEL			
KLM79J	Kingdom of the Netherlands	NaN	37.0
LA SOL NIVEL			
KLM59D	Kingdom of the Netherlands	NaN	0.0
LA SOL NIVEL			
BAW3KG	United Kingdom	-38.099998	263.3
DECOLARE/ATERIZARE COBORARE			
RAM850W	Morocco	NaN	48.2
LA SOL NIVEL			
KLM36E	Kingdom of the Netherlands	NaN	53.7
LA SOL NIVEL			
DLH6JF	Germany	NaN	51.8
LA SOL NIVEL			
TFL6DY	Belgium	487.679993	420.0
DECOLARE/ATERIZARE URCARE			
EIN60G	Ireland	NaN	37.0
LA SOL NIVEL			
CND2J	Kingdom of the Netherlands	NaN	4.6
LA SOL NIVEL			
KLM898	Kingdom of the Netherlands	NaN	9.3
LA SOL NIVEL			
TRA5791	Kingdom of the Netherlands	NaN	29.6
LA SOL NIVEL			
KLM24V	France	NaN	25.0
LA SOL NIVEL			
NUM221	Switzerland	NaN	8.3
LA SOL NIVEL			

Top tari origine:

Kingdom of the Netherlands	8
United States	1
United Kingdom	1
Morocco	1
Germany	1

Bucuresti (LROP)

Nicio aeronavă detectată.

Bucuresti (LROP)

Total : 0 | In aer: 0 | La sol: 0

Aeronave detectate:

Empty DataFrame

Columns: [callsign, origin_country, baro_altitude, velocity_kmh, flight_status, directie]

Index: []

Top tari origine:

=====
Total aeronave procesate in acest batch: 41
=====

=====
MONITORIZARE TRAFIC AERIAN | Batch 5 | 18:00:01
=====

Frankfurt (EDDF)

Total : 15 | In aer: 7 | La sol: 8

Viteza medie : 323.2 km/h

Altitudine medie: 2119 m

In apropiere/aterizare: 9 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	flight_status	directie
LC01511	United States	NaN	0.0		LA
SOL	NIVEL				
DHXCG	Germany	129.539993	9.3		
DECOLARE/ATERIZARE	COBORARE				
DLA8MM	Italy	NaN	35.2		LA
SOL	NIVEL				
RAM810D	Morocco	NaN	0.0		LA
SOL	NIVEL				
ANA223	Japan	76.199997	35.2		LA
SOL	NIVEL				
TWB403	Republic of Korea	922.020020	306.5		
DECOLARE/ATERIZARE	COBORARE				
DLH8AU	Germany	1661.160034	409.8		
APROPIERE	URCARE				
NaN	Germany	NaN	22.2		LA
SOL	NIVEL				
NaN	Germany	99.059998	288.3		
DECOLARE/ATERIZARE	URCARE				
DLH5RX	Germany	76.199997	37.0		LA

SOL	NIVEL				
DLH88H		Germany	76.199997	53.7	LA
SOL	NIVEL				
BAW875		United Kingdom	10980.419922	761.2	
SURVOLARE	NIVEL				
DLH9YW		Germany	655.320007	352.6	
DECOLARE/ATERIZARE		URCARE			
DEWCP		Germany	388.619995	134.8	
DECOLARE/ATERIZARE		NIVEL			
DLA7MT		Italy	NaN	0.0	LA
SOL	NIVEL				

Top tari origine:

Germany	8
Italy	2
United States	1
Morocco	1
Japan	1

Heathrow (EGLL)

Total : 6 | In aer: 5 | La sol: 1
Viteza medie : 325.3 km/h
Altitudine medie: 1128 m
In apropiere/aterizare: 4 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	
flight_status	directie			
SIA308	Singapore	NaN	26.9	LA
SOL	NIVEL			
BAW59FT	United Kingdom	22.860001	240.8	
DECOLARE/ATERIZARE		COBORARE		
RJR818V	Ireland	5356.859863	691.3	
SURVOLARE		COBORARE		
BAW425	United Kingdom	198.119995	233.4	
DECOLARE/ATERIZARE		COBORARE		
BAW91J	United Kingdom	30.480000	238.9	
DECOLARE/ATERIZARE		COBORARE		
BAW749	United Kingdom	30.480000	222.3	
DECOLARE/ATERIZARE		COBORARE		

Top tari origine:

United Kingdom	4
Singapore	1
Ireland	1

Paris CDG (LFPG)

Total : 5 | In aer: 5 | La sol: 0

Viteza medie : 279.8 km/h
Altitudine medie: 319 m
In apropiere/aterizare: 5 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
DILUI	Germany	403.859985	276.7
DECOLARE/ATERIZARE	COBORARE		
AFR804	France	899.159973	360.8
DECOLARE/ATERIZARE	URCARE		
DAH1528	Algeria	30.480000	249.0
DECOLARE/ATERIZARE	COBORARE		
EZY17LA	United Kingdom	121.919998	250.8
DECOLARE/ATERIZARE	COBORARE		
DLH16A	Germany	137.160004	261.9
DECOLARE/ATERIZARE	COBORARE		

Top tari origine:

Germany	2
France	1
Algeria	1
United Kingdom	1

Amsterdam (EHAM)

Total : 18 | In aer: 3 | La sol: 15
Viteza medie : 457.4 km/h
Altitudine medie: 4069 m
In apropiere/aterizare: 2 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
KLM71Y	Kingdom of the Netherlands	NaN	0.0
LA SOL	NIVEL		
KLM617	Kingdom of the Netherlands	NaN	37.0
LA SOL	NIVEL		
N806CH	United States	NaN	21.3
LA SOL	NIVEL		
KLM79J	Kingdom of the Netherlands	NaN	37.0
LA SOL	NIVEL		
KLM59D	Kingdom of the Netherlands	NaN	0.0
LA SOL	NIVEL		
BAW3KG	United Kingdom	NaN	144.5
LA SOL	NIVEL		
RAM850W	Morocco	NaN	48.2
LA SOL	NIVEL		
KLM36E	Kingdom of the Netherlands	NaN	46.3
LA SOL	NIVEL		

DLH6JF		Germany	NaN	51.8
LA SOL	NIVEL			
EIN60G		Ireland	NaN	37.0
LA SOL	NIVEL			
CND2J	Kingdom of the Netherlands		NaN	4.6
LA SOL	NIVEL			
KLM73Z	Kingdom of the Netherlands		NaN	9.3
LA SOL	NIVEL			
KLM898	Kingdom of the Netherlands		NaN	9.3
LA SOL	NIVEL			
TRA5791	Kingdom of the Netherlands	373.380005		340.1
DECOLARE/ATERIZARE	URCARE			
KLM37X	Kingdom of the Netherlands	251.460007		271.3
DECOLARE/ATERIZARE	URCARE			
KLM24V		France	NaN	61.1
LA SOL	NIVEL			
RYR15P		Malta	11582.400391	760.9
SURVOLARE	NIVEL			
NUM221		Switzerland	NaN	8.3
LA SOL	NIVEL			

Top tari origine:

Kingdom of the Netherlands	10
United States	1
United Kingdom	1
Morocco	1
Germany	1

Bucuresti (LROP)

Nicio aeronavă detectată.

Bucuresti (LROP)

Total : 0 | In aer: 0 | La sol: 0

Aeronave detectate:

Empty DataFrame

Columns: [callsign, origin_country, baro_altitude, velocity_kmh, flight_status, directie]

Index: []

Top tari origine:

=====

Total aeronave procesate in acest batch: 44

=====

=====

MONITORIZARE TRAFIC AERIAN | Batch 6 | 18:01:01

Frankfurt (EDDF)

Total : 18 | In aer: 10 | La sol: 8

Viteza medie : 427.1 km/h

Altitudine medie: 3262 m

In apropiere/aterizare: 7 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	
flight_status	directie			
LC01511	United States	NaN	0.0	LA
SOL	NIVEL			
DHXCG	Germany	129.539993	9.3	
DECOLARE/ATERIZARE	COBORARE			
DLA8MM	Italy	NaN	44.5	LA
SOL	NIVEL			
RAM810D	Morocco	NaN	0.0	LA
SOL	NIVEL			
ANA223	Japan	NaN	37.0	LA
SOL	NIVEL			
TWB403	Republic of Korea	662.940002	300.0	
DECOLARE/ATERIZARE	COBORARE			
DLH6VX	Germany	4831.080078	639.7	
SURVOLARE	URCARE			
DLH8AU	Germany	2407.919922	433.4	
APROPIERE	URCARE			
DLH830	Germany	4472.939941	556.5	
SURVOLARE	URCARE			
NaN	Germany	NaN	22.2	LA
SOL	NIVEL			
DLH1LA	Germany	685.799988	339.0	
DECOLARE/ATERIZARE	URCARE			
DLH5RX	Germany	76.199997	40.8	LA
SOL	NIVEL			
DLH88H	Germany	NaN	25.9	LA
SOL	NIVEL			
BAW875	United Kingdom	10980.419922	758.1	
SURVOLARE	NIVEL			
DLH9YW	Germany	1325.880005	393.7	
APROPIERE	URCARE			
DEWCP	Germany	220.979996	113.2	
DECOLARE/ATERIZARE	COBORARE			
EWG3UW	Germany	6903.720215	728.1	
SURVOLARE	COBORARE			
DLA7MT	Italy	NaN	0.0	LA
SOL	NIVEL			

Top tari origine:

Germany	11
Italy	2
United States	1
Morocco	1
Japan	1

Heathrow (EGLL)

Total : 7 | In aer: 6 | La sol: 1
Viteza medie : 368.3 km/h
Altitudine medie: 2117 m
In apropiere/aterizare: 5 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	flight_status	directie
JAL45	Japan	11887.200195	970.8		
SURVOLARE	NIVEL				
SHT19A	United Kingdom	243.839996	242.6		
DECOLARE/ATERIZARE	COBORARE				
SIA308	Singapore	NaN	26.9		LA
SOL	NIVEL				
DLH7H	Germany	487.679993	298.2		
DECOLARE/ATERIZARE	URCARE				
BAW59FT	United Kingdom	22.860001	240.8		
DECOLARE/ATERIZARE	COBORARE				
BAW425	United Kingdom	30.480000	235.2		
DECOLARE/ATERIZARE	COBORARE				
BAW749	United Kingdom	30.480000	222.3		
DECOLARE/ATERIZARE	COBORARE				

Top tari origine:

United Kingdom	4
Japan	1
Singapore	1
Germany	1

Paris CDG (LFPG)

Total : 6 | In aer: 6 | La sol: 0
Viteza medie : 214.7 km/h
Altitudine medie: 124 m
In apropiere/aterizare: 6 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	flight_status	directie
NaN	France	152.399994	74.1		
DECOLARE/ATERIZARE	NIVEL				
DILUI	Germany	205.740005	215.6		

DECOLARE/ATERIZARE COBORARE			
DAH1528	Algeria	30.480000	249.0
DECOLARE/ATERIZARE COBORARE			
AFR41BL	France	137.160004	238.1
DECOLARE/ATERIZARE COBORARE			
EZY17LA	United Kingdom	121.919998	250.8
DECOLARE/ATERIZARE COBORARE			
DLH16A	Germany	99.059998	260.4
DECOLARE/ATERIZARE COBORARE			

Top tari origine:

France	2
Germany	2
Algeria	1
United Kingdom	1

Amsterdam (EHAM)

Total : 14 | In aer: 3 | La sol: 11
Viteza medie : 268.0 km/h
Altitudine medie: 152 m
In apropiere/aterizare: 3 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
flight_status directie			
KLM71Y	Kingdom of the Netherlands	NaN	0.0
LA SOL	NIVEL		
KLM617	Kingdom of the Netherlands	NaN	13.9
LA SOL	NIVEL		
KLM79J	Kingdom of the Netherlands	144.779999	280.0
DECOLARE/ATERIZARE URCARE			
KLM59D	Kingdom of the Netherlands	NaN	0.0
LA SOL	NIVEL		
BAW3KG	United Kingdom	NaN	51.8
LA SOL	NIVEL		
RAM850W	Morocco	NaN	37.0
LA SOL	NIVEL		
KLM36E	Kingdom of the Netherlands	NaN	46.3
LA SOL	NIVEL		
DLH6JF	Germany	NaN	51.8
LA SOL	NIVEL		
EIN60G	Ireland	NaN	37.0
LA SOL	NIVEL		
CND2J	Kingdom of the Netherlands	NaN	4.6
LA SOL	NIVEL		
KLM73Z	Kingdom of the Netherlands	228.600006	258.6
DECOLARE/ATERIZARE URCARE			
KLM1010	Kingdom of the Netherlands	83.820000	265.5
DECOLARE/ATERIZARE COBORARE			

KLM24V	France	NaN	61.1
LA SOL	NIVEL		
NUM221	Switzerland	NaN	8.3
LA SOL	NIVEL		

Top tari origine:

Kingdom of the Netherlands	8
United Kingdom	1
Morocco	1
Germany	1
Ireland	1

Bucuresti (LROP)

Nicio aeronavă detectată.

Bucuresti (LROP)

Total : 0 | In aer: 0 | La sol: 0

Aeronave detectate:

Empty DataFrame

Columns: [callsign, origin_country, baro_altitude, velocity_kmh, flight_status, directie]

Index: []

Top tari origine:

Total aeronave procesate in acest batch: 45
=====

=====

MONITORIZARE TRAFIC AERIAN		Batch 7		18:02:01
----------------------------	--	---------	--	----------

=====

Frankfurt (EDDF)

Total : 16 | In aer: 10 | La sol: 6

Viteza medie : 421.6 km/h

Altitudine medie: 2809 m

In apropiere/aterizare: 8 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
DLH543	Germany	922.020020	305.9
DECOLARE/ATERIZARE	COBORARE		
LC01511	United States	419.100006	337.2
DECOLARE/ATERIZARE	URCARE		
DLA8MM	Italy	76.199997	26.9

LA

SOL	NIVEL				
RAM810D		Morocco	NaN	0.0	LA
SOL	NIVEL				
ANA223		Japan	NaN	38.9	LA
SOL	NIVEL				
TWB403		Republic of Korea	434.339996	245.2	
DECOLARE/ATERIZARE	COBORARE				
DLH830		Germany	4983.479980	618.0	
SURVOLARE	URCARE				
DLH194		Germany	289.559998	267.1	
DECOLARE/ATERIZARE	URCARE				
DLH1LA		Germany	1417.319946	401.6	
APROPIERE	URCARE				
DLH5RX		Germany	NaN	31.5	LA
SOL	NIVEL				
DLH88H		Germany	NaN	10.2	LA
SOL	NIVEL				
BAW875		United Kingdom	10972.799805	751.5	
SURVOLARE	NIVEL				
DLH9YW		Germany	1958.339966	446.0	
APROPIERE	URCARE				
DEWCP		Germany	106.680000	107.6	
DECOLARE/ATERIZARE	COBORARE				
EWG3UW		Germany	6583.680176	735.5	
SURVOLARE	COBORARE				
DLA7MT		Italy	NaN	0.0	LA
SOL	NIVEL				

Top tari origine:

Germany	9
Italy	2
United States	1
Morocco	1
Japan	1

Heathrow (EGLL)

Total : 9 | In aer: 8 | La sol: 1

Viteza medie : 269.3 km/h

Altitudine medie: 325 m

In apropiere/aterizare: 8 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
----------	----------------	---------------	--------------

flight_status	directie
---------------	----------

SHT19A	United Kingdom	38.099998	242.6
--------	----------------	-----------	-------

DECOLARE/ATERIZARE	COBORARE
--------------------	----------

SIA308	Singapore	NaN	26.9	LA
--------	-----------	-----	------	----

SOL	NIVEL
-----	-------

HLE27	United Kingdom	289.559998	201.7
-------	----------------	------------	-------

DECOLARE/ATERIZARE	URCARE		
DLH7H	Germany	1249.680054	472.6
APROPIERE	URCARE		
LOG82WR	United Kingdom	129.539993	209.3
DECOLARE/ATERIZARE	URCARE		
BAW59FT	United Kingdom	22.860001	240.8
DECOLARE/ATERIZARE	COBORARE		
BAW586M	United Kingdom	548.640015	298.3
DECOLARE/ATERIZARE	URCARE		
BAW425	United Kingdom	30.480000	235.2
DECOLARE/ATERIZARE	COBORARE		
BAW513	United Kingdom	289.559998	253.7
DECOLARE/ATERIZARE	COBORARE		

Top tari origine:

United Kingdom	7
Singapore	1
Germany	1

Paris CDG (LFPG)

Total : 6 | In aer: 6 | La sol: 0
Viteza medie : 265.1 km/h
Altitudine medie: 240 m
In apropiere/aterizare: 6 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
flight_status directie			
AFR758	France	800.099976	363.8
DECOLARE/ATERIZARE	URCARE		
DILUI	Germany	76.199997	210.0
DECOLARE/ATERIZARE	COBORARE		
AFR41BL	France	137.160004	238.1
DECOLARE/ATERIZARE	COBORARE		
ITY324	Ireland	205.740005	267.5
DECOLARE/ATERIZARE	COBORARE		
EZY17LA	United Kingdom	121.919998	250.8
DECOLARE/ATERIZARE	COBORARE		
DLH16A	Germany	99.059998	260.4
DECOLARE/ATERIZARE	COBORARE		

Top tari origine:

France	2
Germany	2
Ireland	1
United Kingdom	1

Amsterdam (EHAM)

Total : 13 | In aer: 4 | La sol: 9
Viteza medie : 281.1 km/h
Altitudine medie: 274 m
In apropiere/aterizare: 4 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	flight_status	directie
EJU15EH	Austria	213.360001	256.1		
DECOLARE/ATERIZARE COBORARE					
KLM71Y	Kingdom of the Netherlands	NaN	21.3		
LA SOL	NIVEL				
KLM617	Kingdom of the Netherlands	NaN	7.4		
LA SOL	NIVEL				
KLM79J	Kingdom of the Netherlands	784.859985	438.3		
DECOLARE/ATERIZARE URCARE					
KLM59D	Kingdom of the Netherlands	129.539993	265.0		
DECOLARE/ATERIZARE URCARE					
BAW3KG	United Kingdom	NaN	48.2		
LA SOL	NIVEL				
RAM850W	Morocco	NaN	38.9		
LA SOL	NIVEL				
KLM36E	Kingdom of the Netherlands	NaN	46.3		
LA SOL	NIVEL				
DLH6JF	Germany	NaN	51.8		
LA SOL	NIVEL				
EIN60G	Ireland	NaN	19.4		
LA SOL	NIVEL				
CND2J	Kingdom of the Netherlands	NaN	4.6		
LA SOL	NIVEL				
KLM1010	Kingdom of the Netherlands	-30.480000	165.1		
DECOLARE/ATERIZARE NIVEL					
KLM24V	France	NaN	48.2		
LA SOL	NIVEL				

Top tari origine:

Kingdom of the Netherlands	7
Austria	1
United Kingdom	1
Morocco	1
Germany	1

Bucuresti (LROP)

Total : 1 | In aer: 1 | La sol: 0
Viteza medie : 794.5 km/h
Altitudine medie: 10363 m

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	flight_status
----------	----------------	---------------	--------------	---------------

directie
OMA123 Oman 10363.200195 794.5 SURVOLARE
NIVEL

Top tari origine:
Oman 1

Total aeronave procesate in acest batch: 45

MONITORIZARE TRAFIC AERIAN | Batch 8 | 18:03:01

Frankfurt (EDDF)

Total : 12 | In aer: 6 | La sol: 6
Viteza medie : 312.2 km/h
Altitudine medie: 767 m
In apropiere/aterizare: 7 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	
flight_status	directie			
DLH543	Germany	617.219971	287.0	
DECOLARE/ATERIZARE	COBORARE			
LC01511	United States	967.739990	437.8	
DECOLARE/ATERIZARE	URCARE			
DLA8MM	Italy	NaN	16.7	LA
SOL	NIVEL			
RAM810D	Morocco	NaN	0.0	LA
SOL	NIVEL			
ANA223	Japan	NaN	22.2	LA
SOL	NIVEL			
TWB403	Republic of Korea	236.220001	236.3	
DECOLARE/ATERIZARE	COBORARE			
DLH194	Germany	792.479980	353.8	
DECOLARE/ATERIZARE	URCARE			
DLH1LA	Germany	1882.140015	451.0	
APROPIERE	URCARE			
DLH5RX	Germany	76.199997	31.5	LA
SOL	NIVEL			
DLH88H	Germany	NaN	10.2	LA
SOL	NIVEL			
DEWCP	Germany	106.680000	107.6	
DECOLARE/ATERIZARE	COBORARE			
DLA7MT	Italy	NaN	0.0	LA
SOL	NIVEL			

Top tari origine:

Germany	6
Italy	2
United States	1
Morocco	1
Japan	1

Heathrow (EGLL)

Total : 5 | In aer: 5 | La sol: 0
Viteza medie : 246.2 km/h
Altitudine medie: 145 m
In apropiere/aterizare: 5 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
flight_status directie			
SHT19A	United Kingdom	30.480000	244.5
DECOLARE/ATERIZARE COBORARE			
LOG82WR	United Kingdom	533.400024	260.7
DECOLARE/ATERIZARE URCARE			
BAW59FT	United Kingdom	22.860001	240.8
DECOLARE/ATERIZARE COBORARE			
BAW425	United Kingdom	30.480000	235.2
DECOLARE/ATERIZARE COBORARE			
BAW513	United Kingdom	106.680000	250.0
DECOLARE/ATERIZARE COBORARE			

Top tari origine:

United Kingdom	5
----------------	---

Paris CDG (LFPG)

Total : 6 | In aer: 6 | La sol: 0
Viteza medie : 243.0 km/h
Altitudine medie: 117 m
In apropiere/aterizare: 6 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
flight_status directie			
DILUI	Germany	76.199997	210.0
DECOLARE/ATERIZARE COBORARE			
AFR41BL	France	137.160004	238.1
DECOLARE/ATERIZARE COBORARE			
ITY324	Ireland	121.919998	262.1
DECOLARE/ATERIZARE COBORARE			
EZY17LA	United Kingdom	121.919998	250.8
DECOLARE/ATERIZARE COBORARE			
DLH16A	Germany	99.059998	260.4

DECOLARE/ATERIZARE COBORARE
AFR37NM France 144.779999 236.3
DECOLARE/ATERIZARE COBORARE

Top tari origine:

Germany	2
France	2
Ireland	1
United Kingdom	1

Amsterdam (EHAM)

Total : 13 | In aer: 2 | La sol: 11
Viteza medie : 215.1 km/h
Altitudine medie: -8 m
In apropiere/aterizare: 2 aeronave

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh
flight_status directie			
EJU15EH	Austria	15.24	265.2
DECOLARE/ATERIZARE COBORARE			
KLM71Y Kingdom of the Netherlands		NaN	2.3
LA SOL NIVEL			
KLM617 Kingdom of the Netherlands		NaN	27.8
LA SOL NIVEL			
BAW3KG	United Kingdom	NaN	66.7
LA SOL NIVEL			
RAM850W	Morocco	NaN	38.9
LA SOL NIVEL			
KLM36E Kingdom of the Netherlands		NaN	46.3
LA SOL NIVEL			
NaN	Latvia	NaN	33.3
LA SOL NIVEL			
EIN60G	Ireland	NaN	19.4
LA SOL NIVEL			
CND2J Kingdom of the Netherlands		NaN	4.6
LA SOL NIVEL			
KLM1010 Kingdom of the Netherlands		-30.48	165.1
DECOLARE/ATERIZARE NIVEL			
NaN Kingdom of the Netherlands		NaN	29.6
LA SOL NIVEL			
KLM959 Kingdom of the Netherlands		NaN	10.2
LA SOL NIVEL			
KLM24V	France	NaN	48.2
LA SOL NIVEL			

Top tari origine:

Kingdom of the Netherlands	7
Austria	1

United Kingdom	1
Morocco	1
Latvia	1

Bucuresti (LR0P)

```
-----
Total : 1 | In aer: 1 | La sol: 0
Viteza medie : 601.5 km/h
Altitudine medie: 2537 m
In apropiere/aterizare: 1 aeronave
```

Aeronave detectate:

callsign	origin_country	baro_altitude	velocity_kmh	flight_status
ROT416M	Romania	2537.459961	601.5	APROPIERE
COBORARE				

Top tari origine:

Romania	1
---------	---

Total aeronave procesate in acest batch: 37

7.6. Oprirea streaming-ului

```
# Opreste toate stream-urile active
for q in spark.streams.active:
    print(f"Opresc query-ul: {q.name}")
    q.stop()

print("Toate stream-urile au fost oprite.")
# spark.stop()
```

7.7. Statistici despre topicul Kafka

Citim topicul Kafka în mod batch (nu streaming) pentru a vedea câte mesaje totale au fost publicate de producer de la pornirea sistemului. Aceasta este o metodă utilă pentru a verifica că producer-ul funcționează corect și că mesajele ajung în Kafka.

```
df_count = spark.read \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "zboruri") \
    .option("startingOffsets", "earliest") \
    .load()

total = df_count.count()
print(f"Total mesaje in topicul 'zboruri': {total}")
```